

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHÓA TOÁN-TIN



LUẬN VĂN TỐT NGHIỆP ĐẠI HỌC

**Development of Yolo machine learning
model and real-time streaming operational
parameters of CO₂ micro algae capture pilot**

GIẢNG VIÊN HƯỚNG DẪN:
TS. HOÀNG VĂN HÀ
TS. TRỊNH NGỌC TRUNG

—o0o—

SINH VIÊN THỰC HIỆN:
ĐÀO THANH NGUYỄN-20280068

Lời cam đoan

Tôi xin cam đoan rằng toàn bộ nội dung của khóa luận này là do tôi tự thực hiện dưới sự hướng dẫn của TS. Hoàng Văn Hà và TS. Trịnh Ngọc Trung và không sao chép từ bất kỳ nguồn nào khác ngoài các tài liệu được trích dẫn và tham khảo trong khóa luận. Tôi hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của các nội dung, dữ liệu và các kết quả nghiên cứu trong khóa luận này. Nếu có bất kỳ vi phạm nào về bản quyền hoặc tính xác thực, tôi xin hoàn toàn chịu trách nhiệm.

Lời cảm ơn / Lời ngỏ

Để hoàn thành kì đề cương luận văn này, tôi tỏ lòng biết ơn sâu sắc đến tiến sĩ Trịnh Ngọc Trung và tiến sĩ Hoàng Văn Hà đã hướng dẫn tận tình trong suốt quá trình nghiên cứu.

Tôi chân thành cảm ơn quý thầy, cô trong khoa Toán-Tin, Trường đại học khoa học tự nhiên thành phố Hồ Chí Minh đã tận tình truyền đạt kiến thức trong những năm tôi học tập ở trường.

Cuối cùng, tôi xin chúc quý thầy, cô dồi dào sức khỏe và thành công trong sự nghiệp cao quý.

Tóm tắt nội dung

Nội dung chính của luận văn nhằm tìm hiểu, nghiên cứu xây dựng hệ thống sử dụng các mô hình học máy và điện toán đám mây vào theo dõi quá trình nuôi vi tảo thời gian thực trong công nghiệp dựa trên những công trình, công nghệ mới được nghiên cứu và phát triển trong những năm gần đây. Trong quá trình nghiên cứu, chúng tôi đã tiến hành tổng hợp, đánh giá ưu và nhược điểm của các phương pháp, công nghệ đã và đang được nghiên cứu, sử dụng. Tiếp cận vấn đề theo nhiều hướng khác nhau, chúng tôi thực hiện một số phương pháp sử dụng máy học để dự đoán nồng độ chất, thị giác máy tính để nhận diện bọt khí và xây dựng nền tảng theo dõi thời gian thực trong quá trình nuôi vi tảo. Bên cạnh việc hoàn thành nội dung của đề tài, nhóm chúng tôi đã nghiên cứu thêm một số phần để từ đó đặt nền móng cho các nghiên cứu sau này. Phần còn lại của luận văn tập trung vào việc đánh giá mô hình, xây dựng hệ thống và kết quả đạt được, đồng thời phân tích ưu nhược điểm của mô hình và hệ thống thực hiện và thảo luận những vấn đề mà mô hình và hệ thống còn gặp phải. Cuối cùng, nhóm chúng tôi đề xuất hướng phát triển tiếp theo của đề tài trong tương lai.

Mục lục

1	Giới thiệu tổng quan vấn đề	1
1.1	Đặt vấn đề	1
1.2	Giới thiệu hệ thống truyền phát dữ liệu Raman thời gian thực	2
1.2.1	Kiến trúc	2
1.2.2	Chức năng	3
1.2.3	Ưu điểm	4
1.2.4	Nhược điểm	4
1.3	Giới thiệu hệ thống giám sát bọt khí theo thời gian thực	4
1.3.1	Kiến trúc	5
1.3.2	Chức năng	5
1.3.3	Ưu điểm	5
1.3.4	Nhược điểm	6
1.4	Các mô hình máy học và phương pháp thống kê được sử dụng	6
1.4.1	Hồi quy bình phương tối thiểu riêng phần (Partial Least Squares Regression-PLSR)	6
1.4.2	Tổng quan về Yolov8	6
1.5	Mục tiêu của đề tài	7
1.6	Cấu trúc luận văn	7
2	Xây dựng hệ thống truyền phát tín hiệu Raman thời gian thực	8
2.1	Kiến trúc hệ thống	8
2.1.1	Apache Spark	9
2.1.2	Databricks	11
2.1.3	Delta Lake	12
2.1.4	Event Hubs	13
2.2	Xây dựng hệ thống	14
2.2.1	Triển khai hệ thống tự động bằng code (IaaS)	15
2.2.2	Thu thập dữ liệu	15
2.2.3	Xử lý dữ liệu trong Data Lakehouse	15
2.2.4	Trực quan hóa dữ liệu thời gian thực	16
2.3	Thuật toán	16
2.3.1	Savitzky Golay (S-G) filter	16
2.3.2	Hồi quy bình phương tối thiểu riêng phần (Partial Least Squares Regression - PLSR)	17
3	Mô hình Yolo và ứng dụng nhận diện bọt khí thời gian thực	19
3.1	Kiến trúc hệ thống	19
3.1.1	Ứng dụng chạy mô hình Yolov8 thời gian thực	20

3.1.2	Apache Kafka	21
3.1.3	Telegraf	24
3.1.4	InfluxDB	25
3.1.5	Grafana	26
3.2	Xây dựng hệ thống phát hiện bọt khí thời gian thực	26
3.3	Kiến thức nền tảng trong thị giác máy tính	28
3.3.1	Tích chập (Conv)	28
3.3.2	Bộ lọc (Kernel)	28
3.3.3	Bước nhảy (Stride)	28
3.3.4	Đệm biên (Padding)	28
3.4	Mô hình Yolov8	29
3.4.1	Kiến trúc	29
3.4.2	Tổn thất trong phát hiện đối tượng	32
3.4.3	Hàm tổn thất và quy tắc cập nhật trọng số	32
4	Thực nghiệm và đánh giá	34
4.1	Bộ dữ liệu phổ Raman	34
4.2	Bộ dữ liệu bọt khí trong nuôi vi tảo	35
4.2.1	Cài đặt thiết bị	35
4.2.2	Mô tả bộ dữ liệu	35
4.3	Chỉ số đánh giá mô hình	36
4.3.1	Sai số trung bình tuyệt đối (MAE)	36
4.3.2	Trung bình bình phương sai số (MSE)	36
4.3.3	R^2	37
4.3.4	Độ bao phủ (Recall)	37
4.3.5	Độ chính xác (Precision)	38
4.3.6	F1-score	38
4.3.7	Intersection over union (IoU)	38
4.3.8	Độ chính xác trung bình (mAP)	39
4.4	Kết quả huấn luyện mô hình bình phương tối thiểu riêng phần (PLSR)	39
4.5	Kết quả huấn luyện mô hình Yolov8	40
5	Kết luận	44
5.1	Tóm tắt mục tiêu nghiên cứu	44
5.2	Kết quả và đóng góp chính	44
5.3	Những thử nghiệm và phân tích	45
5.4	Những hạn chế và hướng phát triển	45
5.5	Tầm quan trọng và ứng dụng thực tiễn	45

Danh sách hình vẽ

1.1	Nuôi vi tảo trong công nghiệp.	2
1.2	Hệ thống truyền phát dữ liệu thời gian thực trên nền tảng điện toán đám mây Azure.	3
1.3	Kiến trúc hệ thống nhận diện diện bọt khí.	5
2.1	Thành phần hệ thống.	8
2.2	Kiến trúc spark.	10
2.3	Kiến trúc Databricks.	12
2.4	Bảng lưu trữ trong Delta Lake.	13
2.5	Nhật ký giao dịch.	13
2.6	Kiến trúc Event Hubs.	14
2.7	Sơ đồ hoạt động hệ thống.	15
2.8	Biểu đồ ma trận đầu vào.	17
2.9	Biểu đồ ma trận đầu ra.	18
3.1	Thành phần hệ thống.	19
3.2	Ứng dụng Yolov8 thời gian thực.	20
3.3	Kiến trúc tổng quát của Apache Kafka.	21
3.4	Cấu tạo của một sự kiện.	22
3.5	Tổng quan về chủ đề.	22
3.6	Tính chịu lỗi của topic Info trên 3 brokers, 3 partitions và 3 replications.	23
3.7	Lưu trữ trên Kafka.	23
3.8	Lưu trữ trên Kafka.	24
3.9	Kiến trúc của InfluxDB.	25
3.10	Ví dụ về kernel 3x3.	28
3.11	Ví dụ về stride=1.	29
3.12	Ví dụ về padding 0.	29
3.13	Kiến trúc Yolov8.	30
3.14	Cấu trúc lớp conv.	30
3.15	Cấu trúc lớp Bottleneck.	31
3.16	Cấu trúc lớp C2f.	31
3.17	Cấu trúc Detect.	31
4.1	Biểu đồ về bộ dữ liệu Raman.	34
4.2	Cài đặt thiết bị mô phỏng nuôi vi tảo.	35
4.3	Ví dụ 3 trạng thái trong bộ dữ liệu.	36
4.4	Giá trị thực tế và dự đoán trên tập dữ liệu test.	40
4.5	Quá trình huấn luyện dữ liệu.	40
4.6	Precision và Recall.	41
4.7	Confusion matrix.	42

4.8	Kết quả dự đoán 3 trạng thái trong bộ dữ liệu.	42
5.1	Mô phỏng gửi dữ liệu Raman.	49
5.2	Tạo điểm cuối của mô hình dữ liệu.	50

Chương 1

Giới thiệu tổng quan vấn đề

1.1 Đặt vấn đề

Hiện nay, vi tảo mang lại nhiều lợi ích đáng kể trong cuộc sống, bởi chúng giàu dinh dưỡng và được sử dụng cho nhiều sản phẩm có giá trị cao. Trong quá trình nuôi vi tảo, nhiều chất hóa học có lợi được tạo ra, vì vậy việc tối ưu hóa chất lượng nuôi tảo là cần thiết trong bối cảnh khoa học và công nghệ phát triển. Vi tảo tiêu thụ khí CO₂ để phát triển, do đó, cần sục một lượng khí CO₂ vừa đủ để vi tảo phát triển tốt. Tuy nhiên, quá trình sục khí CO₂ đối mặt với nhiều thách thức, nếu sục khí quá mạnh hoặc quá yếu sẽ ảnh hưởng lớn đến chất lượng vi tảo, thậm chí có thể làm vi tảo ngừng phát triển. Việc kiểm soát lượng khí CO₂ đòi hỏi người nuôi phải giám sát liên tục và điều chỉnh kịp thời để tránh các tình huống xấu có thể xảy ra, gây tổn thất cho tổ chức và doanh nghiệp.

Ngoài ra, tổ chức và doanh nghiệp muốn tận dụng chất hóa học AAA sinh ra trong quá trình nuôi vi tảo để làm nguyên liệu cho các sản phẩm có giá trị cao. Chất AAA này cần được giám sát nồng độ để điều chỉnh các điều kiện nuôi cấy nhằm tạo ra chất với chất lượng tối ưu. Thông thường, để đo được nồng độ chất AAA, cần thuê chuyên gia lấy mẫu và mang về phòng thí nghiệm để đo đạc và tính toán, quá trình này mất từ 1 đến 2 tuần và tiêu tốn nhiều chi phí. Vì thời gian đợi kết quả khá lâu, thường không thể điều chỉnh kịp thời và chất lượng chất AAA thu được sẽ không phải là tốt nhất.

Từ hai vấn đề trên, cần tự động hóa quá trình giám sát chất lượng bọt khí CO₂ và nồng độ chất AAA sinh ra trong nuôi vi tảo mọi lúc, mọi nơi để điều chỉnh kịp thời, đạt chất lượng tối ưu, mang lại lợi ích cho tổ chức và doanh nghiệp. Các công nghệ mạnh mẽ như học máy, thị giác máy tính và nền tảng điện toán đám mây đang giúp việc tối ưu hóa hiệu quả sản xuất và quản lý chất lượng sản phẩm trở nên dễ dàng và hiệu quả hơn.

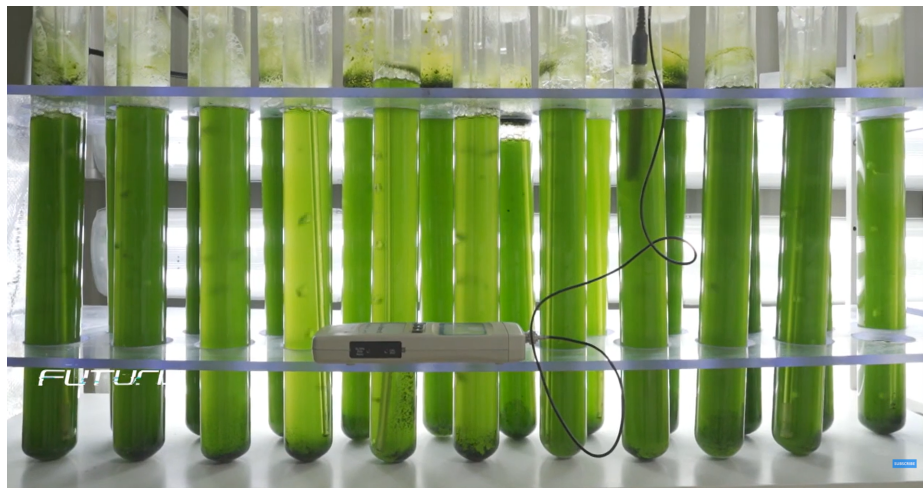
Quá trình nuôi vi tảo giúp tạo ra hóa chất AAA, và bằng cách sử dụng thiết bị Raman để đo tín hiệu điện kết hợp với mô hình học máy, chúng ta có thể dự đoán nồng độ hóa chất AAA một cách chính xác theo thời gian thực. Điều này không chỉ giúp tối ưu hóa quá trình sản xuất mà còn tăng cường khả năng kiểm soát chất lượng sản phẩm một cách linh hoạt và hiệu quả. Một hệ thống truyền phát dữ liệu trực tuyến tích hợp mô hình máy học sẽ mang lại hiệu quả tối đa cho mục tiêu này.

Bằng cách tập trung vào kiểm soát lưu lượng bọt khí CO₂ và nồng độ hóa chất AAA theo thời gian thực trong quá trình nuôi cấy vi tảo, chúng ta có thể đạt được mục tiêu sản xuất một cách nhanh chóng và chính xác mà không cần giám sát trực tiếp tại nhà máy hay phụ thuộc vào các phương pháp phân tích offline tốn kém và mất thời gian. Sự kết hợp giữa mô hình học máy, học sâu và nền tảng điện toán đám mây Azure mở ra khả năng giải quyết các thách thức trong việc tối ưu hóa quá trình vận hành, đồng thời nâng cao tính linh hoạt và chất lượng trong kiểm

soát quá trình sản xuất.

Hiểu rõ những khó khăn trong việc nuôi vi tảo, chúng tôi đã tiến hành xây dựng hệ thống truyền phát dữ liệu thời gian thực trên nền tảng điện toán đám mây Azure và ứng dụng học máy và học sâu để tự động hóa quá trình theo dõi. Chúng tôi triển khai hai hệ thống truyền phát dữ liệu thời gian thực bao gồm: hệ thống theo dõi và dự đoán nồng độ hóa chất AAA dựa trên tín hiệu điện đo từ thiết bị Raman, sử dụng các công cụ lưu trữ và xử lý dữ liệu thời gian thực mạnh mẽ trên hệ thống điện toán đám mây Azure; và hệ thống phát hiện bọt khí CO₂, dự đoán trạng thái sục khí hiện tại trong nuôi vi tảo, mọi dữ liệu sẽ được xử lý và trực quan hóa theo thời gian thực.

Dưới đây, chúng tôi sẽ giới thiệu hệ thống truyền phát dữ liệu thời gian thực trên nền tảng đám mây Azure của Microsoft.



Hình 1.1: Nuôi vi tảo trong công nghiệp.

1.2 Giới thiệu hệ thống truyền phát dữ liệu Raman thời gian thực

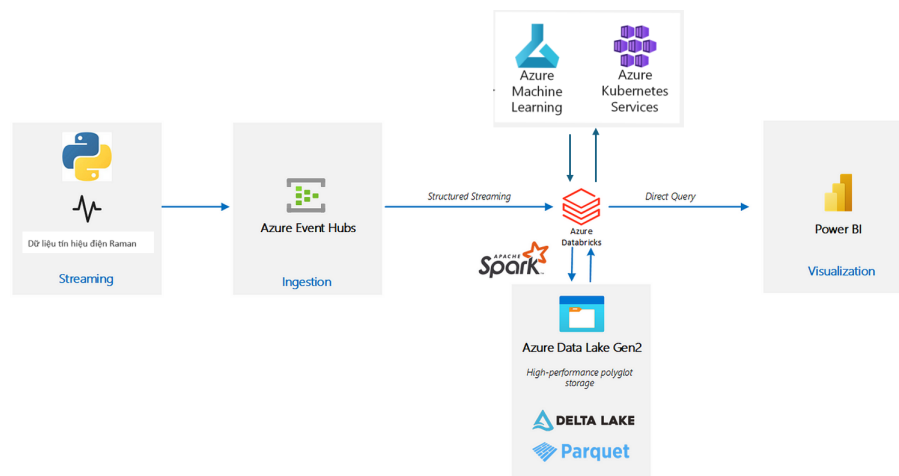
Hệ thống truyền phát dữ liệu thời gian thực trên Azure là một giải pháp công nghệ mạnh mẽ, được thiết kế để xử lý và phân tích dữ liệu trong thời gian thực, mang lại khả năng phản hồi nhanh chóng và hiệu quả cho các ứng dụng và dịch vụ. Sử dụng nền tảng điện toán đám mây Azure, platform này cung cấp một hệ thống linh hoạt và đáng tin cậy cho việc triển khai các giải pháp xử lý dữ liệu thời gian thực. Hệ thống sẽ xử lý tín hiệu điện Raman bằng phương pháp lọc Savitzky Golay (S-G) và tích hợp mô hình học máy bình phương tối thiểu riêng từng phần (Partial Least Squares Regression-PLS) để dự đoán hóa chất AAA.

1.2.1 Kiến trúc

Hệ thống truyền phát dữ liệu thời gian thực trên Azure được xây dựng dựa trên một số thành phần chính sau:

- **Azure Event Hubs:** Đây là dịch vụ xử lý sự kiện quy mô lớn, có khả năng thu thập dữ liệu sự kiện từ hàng triệu thiết bị với độ trễ thấp và băng thông cao. Event Hubs là điểm vào chính cho dữ liệu thời gian thực vào hệ thống.

- **Azure Databricks:** là một nền tảng phân tích dữ liệu dựa trên Apache Spark, nền tảng này cung cấp môi trường giúp xử lý dữ liệu phức tạp, xây dựng và huấn luyện mô hình máy học một cách hiệu quả.
- **Azure Data Lake Gen2:** là một dịch vụ lưu trữ dữ liệu cấp cao. Gen2 kết hợp tính năng của Hadoop Distributed File System (HDFS) với khả năng mở rộng và tính bền vững của Azure Blob Storage, tạo ra một nền tảng lưu trữ dữ liệu linh hoạt và mạnh mẽ.
- **Power BI:** là công cụ trực quan hóa dữ liệu mạnh mẽ của Microsoft, cho phép người dùng kết nối, biến đổi và trực quan hóa dữ liệu từ nhiều nguồn khác nhau một cách dễ dàng. Power BI cung cấp các tính năng trực quan hóa dữ liệu đa dạng như biểu đồ, bảng, bản đồ và dashboard, giúp người dùng hiểu rõ hơn về dữ liệu và đưa ra quyết định thông minh dựa trên thông tin được trực quan hóa một cách rõ ràng.



Hình 1.2: Hệ thống truyền phát dữ liệu thời gian thực trên nền tảng điện toán đám mây Azure.

1.2.2 Chức năng

- **Thu thập dữ liệu thời gian thực:** Từ các thiết bị IoT, ứng dụng web/mobile, và các nguồn dữ liệu khác, đảm bảo dữ liệu được thu thập một cách liên tục và đáng tin cậy.
- **Phân tích dữ liệu thời gian thực:** Phân tích và xử lý dữ liệu ngay lập tức, cho phép phát hiện mẫu, xu hướng và cảnh báo sớm một cách nhanh chóng.
- **Tích hợp và tự động hóa:** Dễ dàng tích hợp với các dịch vụ và ứng dụng khác, tự động hóa các quy trình xử lý dữ liệu và phản hồi.
- **Độ linh hoạt và mở rộng:** Cung cấp khả năng mở rộng tự động, cho phép xử lý lượng dữ liệu lớn mà không cần lo lắng về cơ sở hạ tầng.

Hệ thống truyền phát dữ liệu thời gian thực trên Azure giúp các tổ chức tận dụng sức mạnh của dữ liệu thời gian thực để tối ưu hóa quy trình vận hành, cải thiện trải nghiệm người dùng và đưa ra quyết định kinh doanh một cách nhanh chóng, kịp thời và chính xác. Điều này không chỉ giúp các doanh nghiệp duy trì lợi thế cạnh tranh mà còn tối ưu hóa hiệu suất và giảm thiểu chi phí vận hành.

Nền tảng giúp cung cấp một giải pháp toàn diện cho việc xử lý và phân tích dữ liệu thời gian thực, từ thu thập dữ liệu, xử lý, phân tích, cho đến lưu trữ và trực quan hóa. Điều này giúp các tổ

chức phản ứng nhanh chóng với các sự kiện thời gian thực, tự động hóa quy trình và tạo ra giá trị từ dữ liệu một cách hiệu quả.

1.2.3 Ưu điểm

- **Xử lý dữ liệu thời gian thực:** Hệ thống này cho phép xử lý và phân tích dữ liệu trong thời gian thực, giúp tổ chức phản ứng nhanh chóng với các sự kiện và tình huống cần giải quyết ngay lập tức.
- **Phản hồi nhanh chóng:** Hệ thống truyền phát dữ liệu thời gian thực trên Azure mang lại khả năng phản hồi nhanh chóng và hiệu quả, giúp cải thiện trải nghiệm người dùng và quyết định kinh doanh.
- **Tích hợp linh hoạt:** Hệ thống này dễ dàng tích hợp với các dịch vụ và ứng dụng khác trên nền tảng Azure, tạo ra một hệ sinh thái phân tích dữ liệu toàn diện.
- **Mở rộng dễ dàng:** Hệ thống truyền phát dữ liệu thời gian thực trên Azure cung cấp khả năng mở rộng tự động, cho phép xử lý lượng dữ liệu lớn mà không cần lo lắng về cơ sở hạ tầng.

1.2.4 Nhược điểm

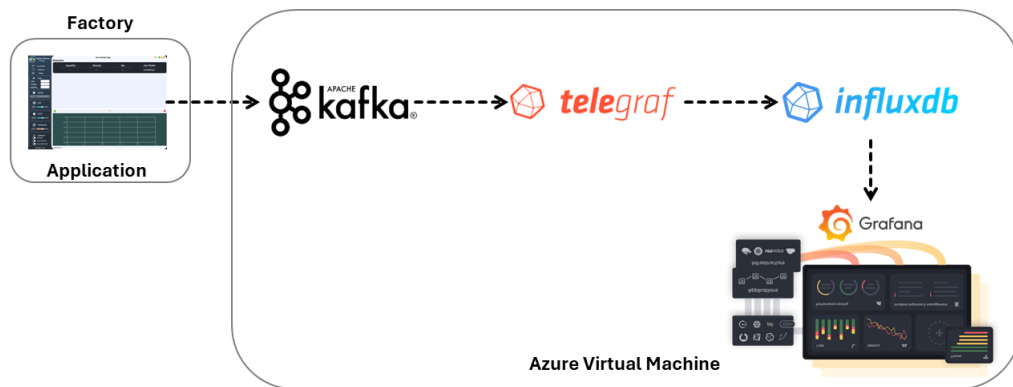
- **Chi phí:** Sử dụng các dịch vụ xử lý dữ liệu thời gian thực có thể tạo ra chi phí cao, đặc biệt khi xử lý lượng dữ liệu lớn và cần sự phản hồi nhanh.
- **Độ phức tạp:** Cấu hình và quản lý hệ thống truyền phát dữ liệu thời gian thực trên Azure có thể đòi hỏi kiến thức chuyên sâu về phân tích dữ liệu và các công nghệ liên quan, đôi khi tạo ra thách thức cho người mới sử dụng.
- **Yêu cầu chuyên môn:** Để tận dụng hết tiềm năng của nền tảng, người dùng cần có kiến thức vững về xử lý dữ liệu thời gian thực và phân tích dữ liệu, có thể tạo ra rào cản cho người mới bắt đầu.

Ngoài hệ thống trên, chúng tôi cũng sẽ giới thiệu hệ thống theo dõi bọt khí CO₂ và trạng thái sức khỏe trong quá trình nuôi vi tảo, sử dụng các công cụ xử lý và lưu trữ mã nguồn mở.

1.3 Giới thiệu hệ thống giám sát bọt khí theo thời gian thực

Như đã đề cập từ đầu chương, nuôi vi tảo cần có hệ thống giám sát chất lượng bọt khí cũng như trạng thái sức khỏe CO₂ là mạnh, yếu hay vừa phải từ đó để giúp người nuôi theo dõi và điều chỉnh phù hợp. Hệ thống giám sát này sẽ sử dụng mô hình học sâu trong thị giác máy tính là YOLOv8 (sẽ đề cập sau) để phát hiện bọt khí CO₂ và trạng thái sức khỏe.

1.3.1 Kiến trúc



Hình 1.3: Kiến trúc hệ thống nhận diện bột khí.

- **Kafka:** là một công cụ xử lý luồng dữ liệu phổ biến nhất hiện nay, được xây dựng để xử lý các luồng dữ liệu lớn một cách nhanh chóng, có khả năng mở rộng và chịu lỗi cao. Kafka được sử dụng rộng rãi cho các hệ thống xử lý thời gian thực và các hệ thống luồng dữ liệu.
- **Telegraf:** là một công cụ dùng để thu thập, xử lý và xuất dữ liệu từ nhiều hệ thống khác nhau. Telegraf hỗ trợ nhiều kết nối đầu vào và đầu ra nên dễ dàng tích hợp với nhiều hệ thống.
- **Influxdb:** là một cơ sở dữ liệu chuỗi thời gian (time-series database) được thiết kế để lưu trữ và truy vấn dữ liệu thời gian thực như sự kiện, logs và metrics. Influxdb được tối ưu trong việc ghi nhận nhanh chóng và truy vấn hiệu quả các dữ liệu theo thời gian thực.
- **Grafana:** là một nền tảng phân tích và trực quan hóa dữ liệu mã nguồn mở, thường được sử dụng để tạo các dashboard tương tác từ nhiều nguồn dữ liệu.

1.3.2 Chức năng

Hệ thống truyền phát dữ liệu thời gian thực từ ứng dụng ở nhà máy tới máy tính người dùng và trực quan hóa dữ liệu giúp người dùng dễ dàng theo dõi tình trạng ngay tức thời.

Hệ thống giúp tự động hóa và dễ dàng tích hợp với các dịch vụ ứng dụng khác. Ngoài ra, hệ thống được xây dựng trên các công cụ mã nguồn mở, nên lâu dài sẽ giúp tối ưu chi phí cho anh nghiệp và đảm bảo an toàn dữ liệu.

1.3.3 Ưu điểm

- Tiết kiệm chi phí trong tương lai và kiểm soát hoàn toàn hệ thống không phụ thuộc vào bên thứ ba.
- Dễ dàng triển khai và tích hợp các dịch vụ và ứng dụng khác.
- Lưu trữ lượng lớn dữ liệu.

1.3.4 Nhược điểm

- Khả năng mở rộng không cao.
- Đòi hỏi đội ngũ kỹ sư quản lý hệ thống phải làm chủ được từng công cụ để kịp thời sửa lỗi và nâng cấp.

1.4 Các mô hình máy học và phương pháp thống kê được sử dụng

Trong hệ thống theo dõi nuôi vi tảo thời gian thực, mô hình máy học dùng để dự đoán tín hiệu điện Raman là mô hình hồi quy bình phương tối thiểu riêng phần được sử dụng trong hệ thống theo dõi nồng độ hóa chất AAA. Bên cạnh đó, mô hình Yolov8 được sử dụng để nhận diện bọt khí và trạng thái sục khí trong nuôi vi tảo, dùng trong hệ thống theo dõi chất lượng bọt khí thời gian thực. Sau đây, chúng tôi sẽ giới thiệu về 2 mô hình trên.

1.4.1 Hồi quy bình phương tối thiểu riêng phần (Partial Least Squares Regression-PLSR)

Hồi quy bình phương tối thiểu riêng phần (PLSR) là một phương pháp phổ biến trong hóa học và công nghệ, dùng để phân tích và dự đoán, thường được áp dụng dưới dạng hồi quy hai khối bình phương tối thiểu riêng phần dự đoán. PLSR liên kết hai ma trận dữ liệu X (dữ liệu đầu vào) và Y (dữ liệu mục tiêu) theo mô hình đa biến tuyến tính, vượt trội hơn so với hồi quy truyền thống bằng cách mô hình hóa cấu trúc của cả X và Y. PLSR có ưu điểm nổi bật nhờ khả năng phân tích dữ liệu chứa nhiều biến nhiễu, có tính cộng tuyến và thậm chí không đầy đủ trong cả X và Y.

Độ chính xác của PLSR ngày càng được cải thiện với sự gia tăng số lượng quan sát. Do đó, PLSR đã trở thành một công cụ tiêu chuẩn trong hóa học và được sử dụng rộng rãi trong cả hóa học và kỹ thuật (xem Svante Wold *et al.*[1]).

1.4.2 Tổng quan về Yolov8

Trong phần này chúng tôi sẽ giới thiệu Yolov8, đây là kỹ thuật chính dùng để nhận diện bọt khí CO₂ và trạng thái sục khí trong quá trình nuôi vi tảo.

Yolo, hay "You Only Look Once", là một thuật toán phát hiện đối tượng trong thị giác máy tính. Yolo đưa ra dự đoán về các hộp giới hạn (bouding box) và xác suất của đối tượng trong một lần truyền hình ảnh duy nhất. Yolo vượt trội so với các thuật toán phát hiện đối tượng khác trong việc đảm bảo tính thời gian thực

Yolov8 là phiên bản mới của mô hình phát hiện đối tượng Yolo. Phiên bản này có kiến trúc giống như các phiên bản tiền nhiệm nhưng có nhiều cải tiến so với các phiên bản trước của Yolo, chẳng hạn như kiến trúc mạng thần kinh mới sử dụng cả Feature Pyra-mid Network (FPN) và Path Aggregation Network(PAN) và một công cụ ghi nhãn mới giúp đơn giản hóa quá trình chú thích. Công cụ ghi nhãn này chứa một số tính năng hữu ích như ghi nhãn tự động, phím tắt ghi nhãn và phím nóng có thể tùy chỉnh. Sự kết hợp của các tính năng này giúp việc chú thích hình ảnh để huấn luyện mô hình trở nên dễ dàng hơn (xem Dillon Reis *et al.*[2]).

FPN hoạt động bằng cách giảm dần độ phân giải không gian của hình ảnh đầu vào đồng thời tăng số lượng kênh đặc trưng. Điều này dẫn đến việc tạo ra các bản đồ đặc trưng có khả năng phát hiện các vật thể ở các tỷ lệ và độ phân giải khác nhau. Mặt khác, kiến trúc PAN tổng hợp

các tính năng từ các cấp độ khác nhau của mạng thông qua việc bỏ qua các kết nối. Bằng cách đó, mạng có thể nắm bắt tốt hơn các đặc điểm ở nhiều tỷ lệ và độ phân giải, điều này rất quan trọng để phát hiện chính xác các vật thể có kích thước và hình dạng khác nhau.

1.5 Mục tiêu của đề tài

Mục tiêu của đề tài là nghiên cứu, hiểu và hiện thực một số phương pháp hiệu quả để xây dựng một hệ thống theo dõi thời gian thực hiệu quả.

Câu hỏi nghiên cứu của đề tài:

- Làm thế nào để giải quyết bài toán trên?
- Cách tiếp cận như thế nào?
- Những công nghệ nào đã và hiện đang được sử dụng?
- Tối ưu hóa chi phí vận hành như thế nào?
- Hướng cải tiến?

Như vậy để thực hiện theo đúng mục tiêu của đề tài cần xác định một số công việc phải giải quyết như sau:

- Tìm kiếm và thu thập dữ liệu phù hợp với nội dung đề tài.
- Tìm hiểu các phương pháp tiếp cận đã được hiện thực
- Lựa chọn mô hình phù hợp
- Lựa chọn kiến trúc hệ thống phù hợp
- Lên kế hoạch hiện thực, phát triển hệ thống theo dõi thời gian thực.

1.6 Cấu trúc luận văn

Nội dung của luận văn sẽ được trình bày trong những chương sau:

- Chương 1: Giới thiệu tổng quan vấn đề
- Chương 2: Xây dựng Hệ thống truyền phát tín hiệu Raman thời gian thực
- Chương 3: Mô hình Yolo và ứng dụng nhận diện bọt khí thời gian thực
- Chương 4: Thực nghiệm và đánh giá
- Chương 5: Kết luận

Chương 2

Xây dựng hệ thống truyền phát tín hiệu Raman thời gian thực

Trong chương này, chúng tôi trình bày chi tiết về kiến trúc hệ thống truyền phát dữ liệu thời gian thực trên nền tảng điện toán đám mây Azure bao gồm các công cụ xử lý và lưu trữ mạnh mẽ hiện nay như Databricks, Azure Blob Storage, Azure Machine Learning, Azure Event Hub và các phương pháp xử lý dữ liệu, thuật toán học máy dùng để dự đoán nồng độ chất AAA sinh ra trong quá trình nuôi vi tảo.

2.1 Kiến trúc hệ thống

Trong chương 1 mục 1.2, chúng ta đã có cái nhìn tổng quan về các thành phần cấu tạo hệ thống. Trong phần này chúng tôi sẽ giới thiệu chi tiết về cấu trúc và nguyên lý hoạt động của các công nghệ sử dụng trong hệ thống.



Hình 2.1: Thành phần hệ thống.

2.1.1 Apache Spark

a) Tổng quan

Apache Spark là một công cụ phân tích phân tán mã nguồn mở để xử lý dữ liệu với khối lượng lớn.

- Hỗ trợ nhiều ngôn ngữ: Apache Spark hỗ trợ nhiều ngôn ngữ; cung cấp API được viết bằng Scala, Java, Python hoặc R. Spark cho phép người dùng viết ra các ứng dụng bằng nhiều ngôn ngữ.
- Tốc độ nhanh: Tính năng quan trọng nhất của Apache Spark là tốc độ xử lý. Spark cho phép ứng dụng chạy trên cụm Hadoop, nhanh hơn một trăm lần trong bộ nhớ và nhanh hơn mười lần trên đĩa.
- Tính toán song song: Spark cung cấp khả năng tính toán song song trên nhiều máy tính cùng một lúc để tăng khả năng xử lý dữ liệu.
- Khả năng chịu lỗi: Spark cung cấp khả năng chịu lỗi khi lưu trữ dữ liệu tính toán trên nhiều máy tính, nó cung cấp khả năng phục hồi khi có một hoặc một vài máy bị lỗi.
- Hoạt động mọi nơi: Spark sẽ chạy trên nhiều nền tảng mà không làm thay đổi tốc độ xử lý. Nó sẽ chạy trên Hadoop, Kubernetes, Mesos, Standalone và thậm chí là cả cloud.
- Một ứng dụng duy nhất: Spark cung cấp rất nhiều thư viện để phục vụ các tính năng: MLlib, DataFrames và SQL cùng với Spark Streaming và GraphX.
- Phân tích nâng cao: Apache Spark cũng hỗ trợ “Map” và “Reduce” đã được đề cập trước đó. Ngoài MapReduce spark còn hỗ trợ Streaming, SQL, Graph và Machine learning. Do đó, Apache Spark có thể được sử dụng để thực hiện phân tích nâng cao.

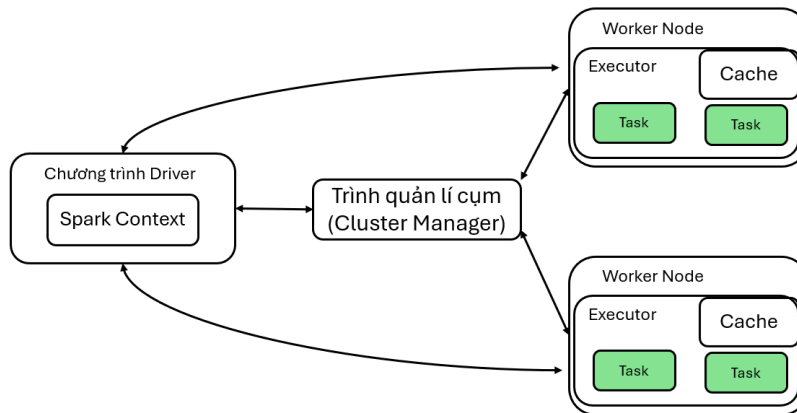
b) Thành phần trong spark

Spark bao gồm 5 thành phần chính:

- Spark Core: Đây là thành phần nền tảng của spark, cung cấp các chức năng I/O cơ bản, lập lịch tác vụ, quản lý bộ nhớ, cơ chế chịu lỗi,...Spark core là nơi chứa API cốt lõi của spark là Resilient Distributed Datasets (RDDs)
- Spark SQL: Được xây dựng dựa trên Spark Core, Spark SQL cho phép truy vấn dữ liệu thông qua SQL và biến thể Apache Hive của SQL, HQL. Spark SQL hỗ trợ các nguồn dữ liệu như bảng Hive, Parquet và JSON, đồng thời có thể kết nối với nhiều cơ sở dữ liệu khác nhau thông qua JDBC, ODBC. Spark SQL cũng là thành phần cung cấp các API được sử dụng nhiều nhất là: Dataframe, Dataset. Đây cũng là thành phần cung cấp khả năng tối ưu hóa quá trình thực thi thông qua API: Catalyst.
- Spark Streaming: Thành phần này cho phép xử lý luồng dữ liệu trực tiếp có khả năng mở rộng và có khả năng chịu lỗi. Dữ liệu có thể được nhập từ nhiều nguồn như Kafka, Flume và Kinesis, có thể được xử lý bằng các thuật toán phức tạp được biểu thị bằng các hàm cấp cao như Map, Reduce, ... Chú ý: Spark Streaming không thực sự là streaming, thực tế nó là một tiến trình vi lô (micro batch)
- MLlib: Thư viện máy học của Spark, MLlib, cung cấp nhiều thuật toán máy học.
- GraphX: Để xử lý đồ thị, GraphX cho phép tạo và thao tác đồ thị cũng như thực hiện các phép tính song song với đồ thị. GraphX cung cấp một tập hợp các toán tử cơ bản và thời gian chạy được tối ưu hóa để tính toán biểu đồ.

c) Kiến trúc

Spark gồm có 2 thành phần chính là: Spark driver (Master), Spark worker node (Slave). Ngoài ra spark còn 1 thành phần nữa giúp spark quản lý cluster là: cluster manager.



Hình 2.2: Kiến trúc spark.

- Spark driver – master
Spark driver là trình điều khiển đóng vai trò điều phối các tác vụ xử lý dữ liệu trên spark cluster.
 - Tạo SparkContext: Spark driver là nơi khởi tạo SparkContext. SparkContext là điểm kết nối tới tài nguyên cụm, cho phép ứng dụng Spark thực hiện các chức năng của mình trên toàn bộ cluster. SparkContext cũng là cầu nối giữa ứng dụng Spark và tài nguyên của Spark cluster. SparkContext hoạt động trên một tiến trình Java duy nhất
 - Điều phối task: Spark driver chia ứng dụng thành từng đơn vị nhỏ hơn để thực hiện trên các worker node.
 - Phân công task: Spark driver không thực hiện các task mà nó đã tạo ra mà đẩy các task vụ này xuống các worker node.
 - Nhận thông tin worker node: Spark driver nhận thông tin về trạng thái và kết quả các task đang thực hiện.
 - Quản lý ứng dụng: Tạo và gửi các biến cục bộ đến các worker node, xử lý lỗi và các ngoại lệ.
- Spark worker node – slave
Spark worker node là máy chủ mà tại đó các task (tính toán, lưu trữ) sẽ được thực thi.
 - Lưu trữ các executor: Spark worker node chính là nơi thực hiện các task được driver gửi tới thông qua các trình thực thi (executor).
 - Cung cấp tài nguyên: Spark worker node cũng chính là tài nguyên chính của Spark cluster.
 - Trao đổi với Spark driver: Spark worker node thực hiện giao tiếp với driver để cập nhật trạng thái cũng như kết quả đến cho driver.
- Executor
Spark executor là một tiến trình java nằm trên các worker node, đây chính là thành phần trực tiếp thực hiện các task được driver gửi tới.
 - Thực thi task: Trình thực thi (Spark executor) chịu trách nhiệm thực thi task.

-
- Lưu trữ dữ liệu: Trình thực thi (Spark executor) không chỉ thực hiện tính toán mà còn quản lý dữ liệu cần thiết cho các tác vụ đó. Đây là nơi có thể lưu trữ cache data và shuffle data.
 - Vòng đời: Spark executor thường được khởi chạy khi bắt đầu ứng dụng Spark và duy trì hoạt động trong suốt thời gian của ứng dụng.

2.1.2 Databricks

Databricks là một nền tảng phân tích thống nhất cho việc xây dựng, triển khai, chia sẻ và duy trì các giải pháp dữ liệu, phân tích và trí tuệ nhân tạo của doanh nghiệp ở quy mô lớn. Nền tảng thông tin dữ liệu của Databricks tích hợp lưu trữ đám mây và bảo mật trong tài khoản đám mây của chúng ta như Azure, AWS, Google Cloud.

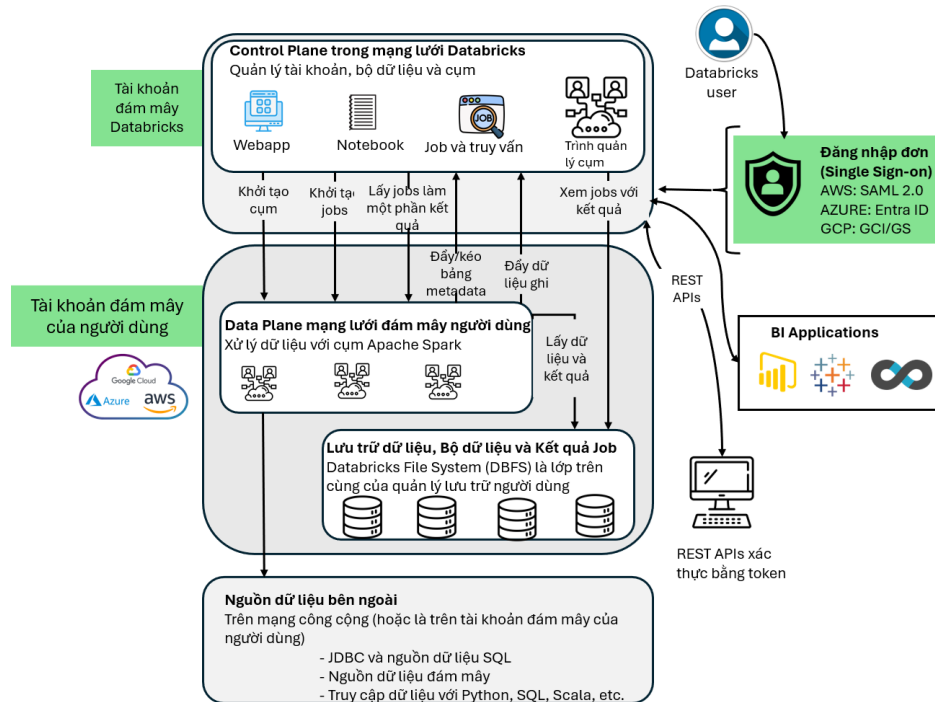
a) Một số công nghệ thành phần:

- **Apache Spark:** Databricks về bản chất là một dịch vụ quản lý cho Spark và là một thành phần cốt lõi của hệ sinh thái Databricks.
- **Workspace:** chứa tất cả các tài nguyên và thư viện được sử dụng trong môi trường Databricks; các công việc cụ thể được chạy thông qua các notebooks (tương tự như các công cụ như Jupyter hoặc Google Colab). Tài nguyên tính toán được phân bổ thông qua Runtimes (đề cập sau).
- **Databricks AutoML:** là một dịch vụ cho phép chúng ta xây dựng các mô hình học máy trong một môi trường low-code. MLflow theo dõi các thử nghiệm học máy bằng cách ghi các thông số, chỉ số, phiên bản dữ liệu và mã, và bất kỳ ý tưởng mô hình nào từ quá trình đào tạo. Thông tin huấn luyện sau đó so sánh các lần chạy trước đó và để thực hiện một bài kiểm tra.
- **Delta Lake:** cung cấp một lớp định dạng bảng trên hồ dữ liệu (Data Lake) cung cấp cho chúng ta một cái nhìn giống như cơ sở dữ liệu quan hệ và giao dịch ACID thông qua nhật ký được liên kết với mỗi bảng Delta. Trong khi Delta Lake tối ưu hóa lớp lưu trữ, việc truy cập dữ liệu trong Databricks vẫn yêu cầu viết mã Spark trong hầu hết các trường hợp sử dụng hiện tại. Delta Live Tables hoạt động như một lớp quản lý đường ống làm việc với Delta Lake để kiểm tra tự động, cũng như giám sát và khả năng quan sát tốt hơn cho các đường ống xử lý dữ liệu.
- **Unity Catalog:** được sử dụng như một mục lục (catalog) quản trị trung tâm cho siêu dữ liệu và quản lý người dùng. Về mặt khái niệm, Unity Catalog cho phép chúng ta quản lý quyền truy cập của người dùng trên các bảng, thay vì trên các tệp trong hồ dữ liệu. Điều này được thực hiện thông qua SQL Grants. Điều này khác biệt so với Delta Lake, nơi cung cấp một cái nhìn về mô hình của các tệp dữ liệu, nhưng cả hai làm việc cùng nhau.
- **Một Runtime:** về cơ bản là một cụm Spark cung cấp các tài nguyên tính toán để chạy một công việc cụ thể. Các loại Runtime có sẵn bao gồm:
Cụm công việc (Worker cluster) - được cung cấp để chạy một notebook cụ thể. Có thể được đặt để tự động mở rộng hoặc gọi qua API. Pools - cho phép chúng ta đặt trước một số lượng tài nguyên nhất định, vì notebook của chúng ta sẽ chạy thường xuyên.

b) Kiến trúc

Kiến trúc của Databricks là một phương pháp đơn giản và tinh tế, hoàn toàn phù hợp với môi trường đám mây (và chỉ dành cho đám mây), kết hợp mượt mà nền tảng đám mây Databricks

với tài khoản đám mây AWS, Google hoặc Azure hiện có. Tận dụng các tùy chọn mã nguồn mở, Databricks linh hoạt hỗ trợ các phương thức đa dạng để thực hiện chiến lược dữ liệu của doanh nghiệp, tất cả trong một nền tảng thống nhất, mượt mà và hiệu quả.



Hình 2.3: Kiến trúc Databricks.

Hai thành phần chính của Kiến trúc Databricks là trình điều khiển (Control Plane) và trình dữ liệu (Data Plane).

- Control Plane đặt các dịch vụ bên dưới (backend) của Databricks, bao gồm giao diện đồ họa và REST APIs để quản lý tài khoản và không gian làm việc.
- Data Plane xử lý tương tác và xử lý dữ liệu từ bên ngoài.

Đáng lưu ý là trong khi phương pháp thông thường là tận dụng tài khoản đám mây (ví dụ: AWS, Azure hoặc GCP) cho cả Data Plane và lưu trữ dữ liệu, Databricks cũng hỗ trợ một kiến trúc trong đó Data Plane tồn tại trong đám mây của chúng ta và chỉ lưu trữ dữ liệu trong tài khoản đám mây của chúng ta.

Bảo mật là một yếu tố chính của kiến trúc Databricks, với các tính năng như mã hóa, kiểm soát truy cập, quản lý dữ liệu và các điều khiển bảo mật kiến trúc được triển khai để đảm bảo bảo vệ và tính toàn vẹn của dữ liệu.

Kiến trúc bảo mật của Databricks được thiết kế để cung cấp các biện pháp toàn diện để bảo vệ dữ liệu và đảm bảo tính toàn vẹn của nền tảng. Kiến trúc tích hợp các tính năng bảo mật và các phương pháp tốt nhất để bảo vệ thông tin nhạy cảm và ngăn chặn truy cập không được ủy quyền.

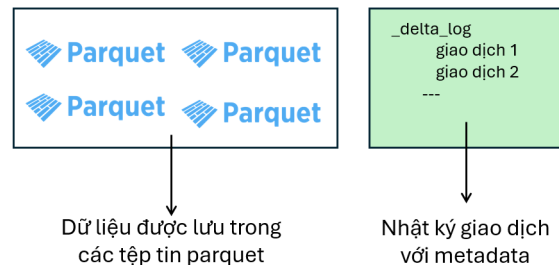
2.1.3 Delta Lake

Delta Lake là một lớp lưu trữ mã nguồn mở, mang đến giao dịch ACID (đề cập bên dưới), bảo vệ các lược đồ (schema) và xử lý công việc dữ liệu lớn. Delta Lake được thiết kế để cải

thiện và mở rộng các tính năng của Apache Spark, đặc biệt là trong việc xử lý dữ liệu lớn, chịu tải cao và thực hiện các thao tác dữ liệu cùng lúc (concurrency).

Điều này cho phép Delta Lake hỗ trợ việc đọc và ghi đồng thời (concurrent read-write), điều mà các định dạng lưu trữ dữ liệu truyền thống không hỗ trợ.

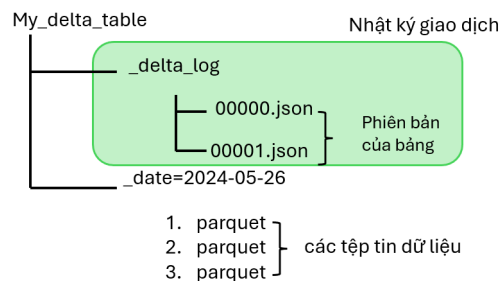
a) Bảng lưu trữ



Hình 2.4: Bảng lưu trữ trong Delta Lake.

Trong Delta Lake, một Bảng là một tập hợp dữ liệu được tổ chức vào một lược đồ và cấu trúc thư mục. Nó đại diện cho đơn vị lưu trữ cơ bản. Dữ liệu sẽ được lưu vào định dạng file Parquet, đây là một định dạng file sử dụng trong Hadoop Distributed File System (HDFS) giúp tối ưu lưu trữ và tăng tốc độ truy vấn.

c) Nhật ký giao dịch: Đảm bảo tuân thủ ACID



Hình 2.5: Nhật ký giao dịch.

Delta Lake hỗ trợ các giao dịch ACID (Atomicity, Consistency, Isolation, Durability) để đảm bảo tính toàn vẹn và đáng tin cậy của dữ liệu trong quá trình xử lý. ACID là viết tắt của Atomicity (Nguyên tử), Consistency (Nhất quán), Isolation (Độc lập) và Durability (Bền vững). Đây là các tính năng quan trọng của hệ thống quản lý cơ sở dữ liệu (DBMS) để đảm bảo tính toàn vẹn và đáng tin cậy của dữ liệu trong quá trình thực hiện các giao dịch.

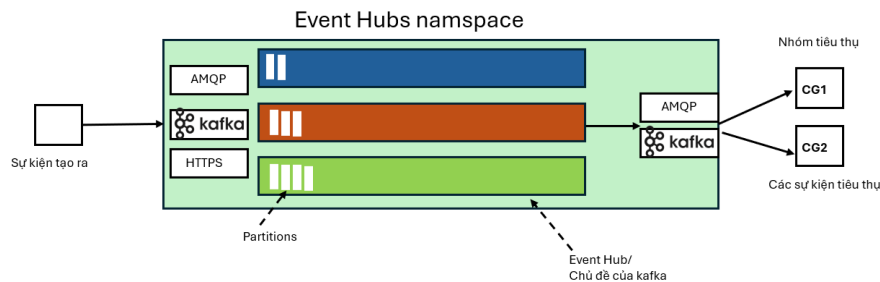
2.1.4 Event Hubs

Azure Event Hubs là một dịch vụ lưu trữ dữ liệu theo dạng luồng dữ liệu được xây dựng trên nền tảng đám mây, có thể truyền hàng triệu sự kiện mỗi giây, với độ trễ thấp, từ bất kỳ nguồn nào đến bất kỳ điểm đến nào. Event Hubs tương thích với Apache Kafka, và cho phép chúng ta chạy các công việc Kafka hiện có mà không cần thay đổi mã nguồn.

Bằng cách sử dụng Event Hubs để tiếp nhận và lưu trữ dữ liệu dạng luồng, các doanh nghiệp có thể tận dụng sức mạnh của dữ liệu dạng luồng để đạt được những thông tin quý báu, thực hiện phân tích thời gian thực và phản ứng với các sự kiện khi chúng xảy ra, từ đó nâng cao hiệu quả tổng thể và trải nghiệm.

Event Hubs cung cấp một nền tảng luồng sự kiện thống nhất với bộ đệm lưu trữ thời gian, giảm bớt sự liên kết giữa nhà sản xuất sự kiện và ứng dụng tiêu thụ sự kiện. Các nhà sản xuất và ứng dụng tiêu thụ có thể thực hiện việc nhập dữ liệu quy mô lớn thông qua nhiều giao thức.

Hình dưới đây mô tả các thành phần chính của kiến trúc Event Hubs:



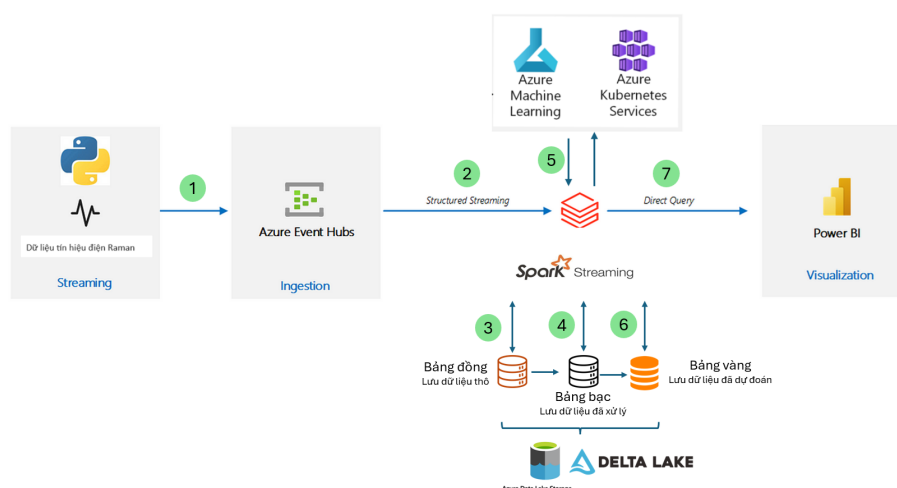
Hình 2.6: Kiến trúc Event Hubs.

Các thành phần chức năng chính của Event Hubs bao gồm:

- **Các ứng dụng nhà sản xuất:** Các ứng dụng này có thể nhập dữ liệu vào một trung tâm sự kiện bằng cách sử dụng các SDK của Event Hubs.
- **Namespace:** Đây là thùng chứa (container) quản lý cho một hoặc nhiều trung tâm sự kiện hoặc chủ đề Kafka. Các nhiệm vụ quản lý như phân bổ khả năng truyền luồng, cấu hình bảo mật mạng, kích hoạt khôi phục sau thảm họa địa lý, v.v. được xử lý ở mức namespace.
- **Trung tâm sự kiện/Chủ đề Kafka:** Trong Event Hubs, chúng ta có thể tổ chức các sự kiện thành một trung tâm sự kiện hoặc một chủ đề Kafka. Đó là một bản ghi phân tán chỉ được ghi thêm, có thể bao gồm một hoặc nhiều phân vùng.
- **Phân vùng:** Các phân vùng được sử dụng để mở rộng một trung tâm sự kiện. Chúng tương tự như các làn đường trên một con đường cao tốc. Nếu chúng ta cần nhiều lưu lượng truyền luồng hơn, chúng ta cần thêm các phân vùng.
- **Các ứng dụng tiêu thụ:** Các ứng dụng này tiêu thụ dữ liệu bằng cách tìm kiếm thông qua nhật ký sự kiện và duy trì bộ điều chỉnh của người tiêu thụ. Người tiêu thụ có thể là tiêu thụ (consumer) Kafka hoặc khách hàng (client) SDK Event Hubs.
- **Nhóm Tiêu thụ:** Đây là một nhóm logic các phiên tiêu thụ đọc dữ liệu từ một trung tâm sự kiện/Chủ đề Kafka, cho phép nhiều người tiêu thụ đọc dữ liệu truyền luồng giống nhau trong một trung tâm sự kiện độc lập với tốc độ riêng và với các bộ điều chỉnh riêng.

2.2 Xây dựng hệ thống

Sau khi đã tìm hiểu cụ thể từng công nghệ, chúng tôi bắt tay vào việc xây dựng một hệ thống dựa vào các công nghệ đã trình bày ở trên. Hệ thống nhận dữ liệu đầu vào là tín hiệu điện Raman và đầu ra hệ thống là đồ thị thời gian thực biểu diễn dữ liệu đầu vào và kết quả dự đoán bằng mô hình máy học.



Hình 2.7: Sơ đồ hoạt động hệ thống.

Hệ thống được xây dựng dựa theo kiến trúc Data lakehouse bằng nền tảng Databricks. Kiến trúc này hiện đang là giúp đáp ứng các yêu cầu hiện nay của doanh nghiệp về lưu trữ và xử lý dữ liệu, cũng như dễ dàng quản lý và mở rộng.

2.2.1 Triển khai hệ thống tự động bằng code (IaaC)

Terraform là một công cụ mạnh mẽ để quản lý cơ sở hạ tầng dưới dạng mã, cho phép chúng ta triển khai và quản lý các tài nguyên trên nhiều nhà cung cấp dịch vụ đám mây, bao gồm Azure. Chúng tôi thực hiện sử dụng Terraform để triển khai các dịch vụ Azure Cloud như Event Hub, Databricks, Azure Data Storage và Azure Machine Learning.

2.2.2 Thu thập dữ liệu

Hệ thống này, chúng tôi thực hiện đọc dữ liệu từ bộ dữ liệu tín hiệu điện Raman và truyền dữ liệu vào Azure Event Hubs đóng vai trò như một hàng đợi tin nhắn/sự kiện. Sau đó, dùng Spark Streaming trong Databricks đọc dữ liệu trực tuyến từ Azure Event Hubs.

2.2.3 Xử lý dữ liệu trong Data Lakehouse

Ngay sau khi quá trình đọc dữ liệu là quá trình ghi dữ liệu vào bảng đồng, nơi dùng để lưu trữ dữ liệu thô. Thực hiện tiền xử lý dữ liệu chẳng hạn như chuyển đổi kiểu dữ liệu phù hợp, xử lý dữ liệu thiếu, lọc cột cần thiết cho mô hình máy học. Bước tiếp theo là lưu các dữ liệu đã xử lý vào bảng bạc. Trong Azure Machine Learning, chúng tôi triển khai mô hình máy học (mô hình bình phương tối thiểu riêng từng phần) đã được huấn luyện thành một dịch vụ điểm cuối. Dùng Spark Streaming chuyển đổi dữ liệu phù hợp với yêu cầu đầu vào mô hình học máy (sử dụng Savitzky Golay (S-G) filter), gọi điểm cuối của Azure Machine Learning và truyền vào dữ liệu đã qua bước tiền xử lý sẽ thu được kết quả dự đoán theo thời gian thực và lưu dữ liệu đã dự đoán xuống bảng vàng. Bảng vàng sẽ dùng cho việc trực quan hóa dữ liệu thời gian thực và phân tích dữ liệu lịch sử. Nhờ có Databricks, chúng tôi có thể triển khai kiến trúc Data Lakehouse hiện đang là xu hướng hiện tại trong việc xây dựng hệ thống dữ liệu lớn trên thế giới.

2.2.4 Trục quan hóa dữ liệu thời gian thực

Chúng tôi sử dụng phương pháp truy vấn trực tiếp để truyền dữ liệu trực tuyến cho kho dữ liệu thời gian thực trên Power BI nhằm tối ưu tốc độ truyền phát, đây hiện là phương thức truyền dữ liệu tối ưu tốc độ nhất khi thực hiện truyền phát dữ liệu tới dịch vụ Power BI. Đối với nhà khoa học dữ liệu hay nhà phân tích dữ liệu muốn phân tích dữ liệu lịch sử, ta có thể sử dụng kết nối từ bảng Delta Lake tới Power BI desktop để thực hiện truy vấn và phân tích.

Tiếp theo, chúng tôi sẽ trình bày thuật toán chuyển đổi dữ liệu và thuật toán máy học đã sử dụng trong hệ thống.

2.3 Thuật toán

Trong chương 1 mục 1.4, chúng ta đã có cái nhìn tổng quan về các mô hình được sử dụng trong hệ thống. Trong phần này chúng tôi sẽ giới thiệu chi tiết về mô hình hồi quy bình phương tối thiểu riêng phần và hoạt động của thuật toán. Ngoài ra, chúng tôi còn sử dụng thêm thuật toán Savitzky Golay (S-G) dùng để tiền xử lý dữ liệu trước khi đưa vào mô hình, vì vậy chúng tôi cũng sẽ trình bày thêm thuật toán này.

2.3.1 Savitzky Golay (S-G) filter

Savitzky Golay (S-G) filter là phương pháp giữ lại hình dạng tín hiệu gốc và do đó, thông tin của nó trong khi vẫn sử dụng các nguyên lý của bộ lọc trung bình trượt. Bộ lọc trung bình trượt là một trong những cách đơn giản và trực tiếp nhất để lọc một tín hiệu nhiễu. Công thức tổng quát của quá trình lặp lại của việc tính trung bình để lọc tín hiệu này được biểu diễn như sau:

$$g_k = \sum_{i=k-m}^{k+m} C_i X_k, \quad (2.1)$$

Trong đó:

- C_i là hệ số lọc với giá trị hằng số bằng 1.
- X_k là một điểm ngẫu nhiên của một tập dữ liệu rời rạc.
- m là số tính.

Để làm điều này, cần thay thế C_i trong công thức 2.1 bằng đa thức bậc cao hơn. Một phương pháp được đề xuất là xấp xỉ đa thức bậc tối thiểu cục bộ phù hợp với đường thẳng đa thức với $n!$ điểm. Tiêu chí được sử dụng trong việc chọn giá trị đã lọc g_k bằng cách xem xét giá trị nào giữ hình dạng cơ bản của dữ liệu tốt nhất; các hệ số của mỗi đa thức phải được xác định sao cho đường cong đa thức tương đương phù hợp nhất với dữ liệu được cung cấp.

Ý tưởng là tìm kiếm sự phù hợp nhất của đa thức có bậc p thông qua một tập hợp $2m + 1$ giá trị liên tục, trong đó $p < 2m + 1$.

$$g_k = \sum_{i=p}^0 b_{pi} k^i. \quad (2.2)$$

Lấy đạo hàm bậc một và hai của công thức 2.2, ta được:

$$\frac{dg_k}{dk} = b_{p1} + 2b_{p2}k + 3b_{p3}k^2 + \dots + pb_{pp}k^{p-1}. \quad (2.3)$$

Tổng quát:

$$\frac{d^p g_k}{dk^p} = pb_{pp}. \quad (2.4)$$

Theo tiêu chí bình phương ít nhất, cần tối thiểu hóa tổng của bình phương của sự khác biệt giữa các giá trị quan sát y_k và các giá trị ước lượng bên trong cửa sổ, do đó:

$$\frac{\partial}{\partial b_{pi}} \left[\sum_{k=-m}^m (g_k - y_k)^2 \right] = 0. \quad (2.5)$$

Biểu diễn Công thức 2.5 liên quan đến b_{pk} ta có:

$$\frac{\partial}{\partial b_{pk}} \left[\sum_{k=-m}^m (b_{p0} + b_{p1}k + \dots + b_{pp}k^p - y_k)^2 \right] = 2 \sum_{k=-m}^m (b_{p0} + b_{p1}k + \dots + b_{pp}k^p - y_k) k = 0. \quad (2.6)$$

Cuối cùng, bộ lọc S-G có một tính chất quan trọng là bảo toàn những đặc trưng quan trọng, rất hữu ích trong phân tích tín hiệu (xem Francis Chinomso Maduakolam *et al.* [4]).

2.3.2 Hồi quy bình phương tối thiểu riêng phần (Partial Least Squares Regression - PLSR)

Hồi quy bình phương tối thiểu riêng phần chủ yếu được phát triển vào đầu những năm 1980 bởi các nhà hóa học tính toán người Scandinavia là Svante Wold và Harald Martens. Nền tảng lý thuyết dựa trên khung mô hình bình phương tối thiểu riêng phần của Herman Wold (cha của Svante).

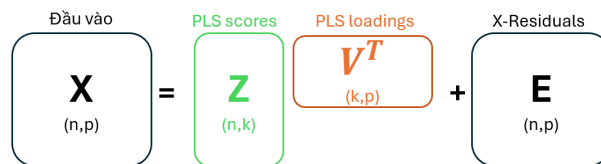
Hồi quy bình phương tối thiểu riêng phần được phát triển như một giải pháp thuật toán không có tiêu chí tối ưu hóa được xác định rõ ràng. Phương pháp được giới thiệu trong lĩnh vực hóa học tính toán và từ từ thu hút sự chú ý của các nhà thống kê ứng dụng.

a) Mô hình hồi quy bình phương tối thiểu riêng phần

Trong hồi quy bình phương tối thiểu riêng phần, chúng ta tìm kiếm các thành phần $z_1, \dots, z_k$, là các tổ hợp tuyến tính của các đầu vào (ví dụ $z_1 = Xw_1$). Trong mô hình hồi quy bình phương tối thiểu riêng phần chúng ta muốn thu được các thành phần sao cho chúng là những dự đoán tốt cho cả kết quả y cũng như các đầu vào x_j cho $j = 1, \dots, p$.

Hơn nữa, có một giả định ngầm trong mô hình hồi quy PLS: cả X và y được giả định là các hàm của một số lượng thành phần giảm ($k < p$) $Z = [z_1, \dots, z_k]$ có thể được sử dụng để phân rã các đầu vào và phản hồi như sau:

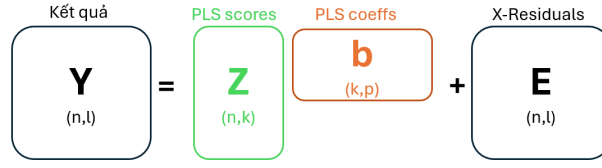
$$X = ZV^T + E.$$



Hình 2.8: Biểu đồ ma trận đầu vào.

và

$$y = Zb + e.$$



Hình 2.9: Biểu đồ ma trận đầu ra.

nơi mà:

- Z là ma trận của các thành phần PLS
- V là ma trận của các tải trọng PLS
- E là ma trận của các dư X
- b là vector của các hệ số hồi quy PLS
- e là vector của các dư y

Bằng cách lấy một số lượng ít các thành phần $Z = [z_1, \dots, z_k]$, chúng ta có thể có các xấp xỉ cho các đầu vào và phản hồi. Mô hình cho không gian X là:

$$\hat{X} = ZV^T.$$

Lần lượt, mô hình dự đoán cho y được cho bởi:

$$\hat{y} = Zb_{pls}.$$

các hệ số b thu được là những hệ số liên quan đến các thành phần PLS.

b) Thuật toán hồi quy bình phương tối thiểu riêng phần

Chúng ta giả định các biến X và y đã được chuẩn hóa về trung bình và tiêu chuẩn hóa.

1. Chúng ta bắt đầu bằng cách đặt $X_0 = X$, và $y_0 = y$.
2. Lặp lại cho $h = 1, \dots, r = \text{rank}(X)$:
3. Bắt đầu với trọng số $\tilde{w}_h = \frac{X_{h-1}^T y_{h-1}}{y_{h-1}^T y_{h-1}}$.
4. Chuẩn hóa trọng số: $w_h = \frac{\tilde{w}_h}{|\tilde{w}_h|}$.
5. Tính toán thành phần PLS: $z_h = \frac{X_{h-1} w_h}{w_h^T w_h}$.
6. Hồi quy y_h và z_h để tìm hệ số hồi quy: $b_h = \frac{y_h^T z_h}{z_h^T z_h}$.
7. Hồi quy tất cả x_j lên z_h : $v_h = \frac{X_{h-1}^T z_h}{z_h^T z_h}$.
8. cập nhật X bằng cách loại bỏ ảnh hưởng của thành phần vừa tìm thấy:
 $X_h = X_{h-1} - z_h v_h^T$.
9. cập nhật Y bằng cách loại bỏ ảnh hưởng của thành phần vừa tìm thấy: $y_h = y_{h-1} - b_h z_h$.

PLS đang làm là tính toán tất cả các thành phần khác nhau (ví dụ: w_h, z_h, v_h, b_h) một cách riêng biệt, sử dụng hồi quy bình phương nhỏ nhất.

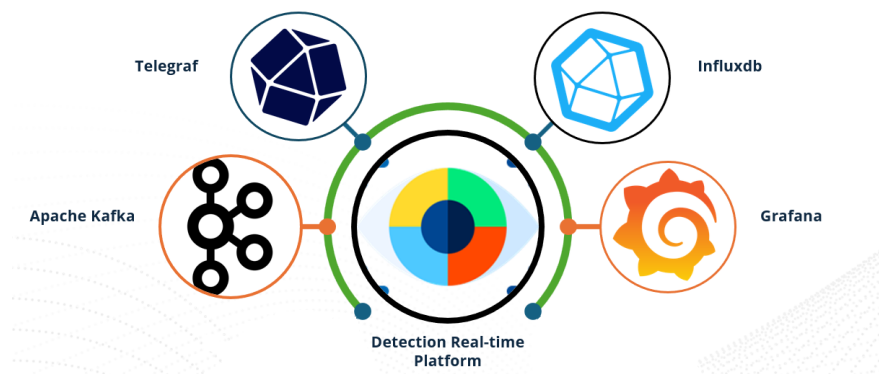
Chương 3

Mô hình Yolo và ứng dụng nhận diện bọt khí thời gian thực

Trong chương này, chúng tôi sẽ trình bày các công nghệ được sử dụng trong hệ thống phát hiện bọt khí thời gian thực và thuật toán Yolov8 sử dụng phát hiện bọt khí và trạng thái môi trường (thấp, bình thường, cao). Bằng cách hiểu rõ từng thành phần trong hệ thống, chúng ta có thể xây dựng một hệ thống thời gian thực hiệu quả và tối ưu.

3.1 Kiến trúc hệ thống

Hệ thống phát hiện bọt khí sử dụng Yolov8 để phát hiện bọt khí từ hình ảnh, video hoặc camera, Kafka để xử lý dữ liệu nhanh chóng, Telegraf để thu thập và gửi dữ liệu vào InfluxDB, và Grafana để trực quan hóa dữ liệu trong thời gian thực. Sự kết hợp này tạo nên một giải pháp mạnh mẽ và hiệu quả cho việc giám sát và phân tích bọt khí trong các ứng dụng công nghiệp.

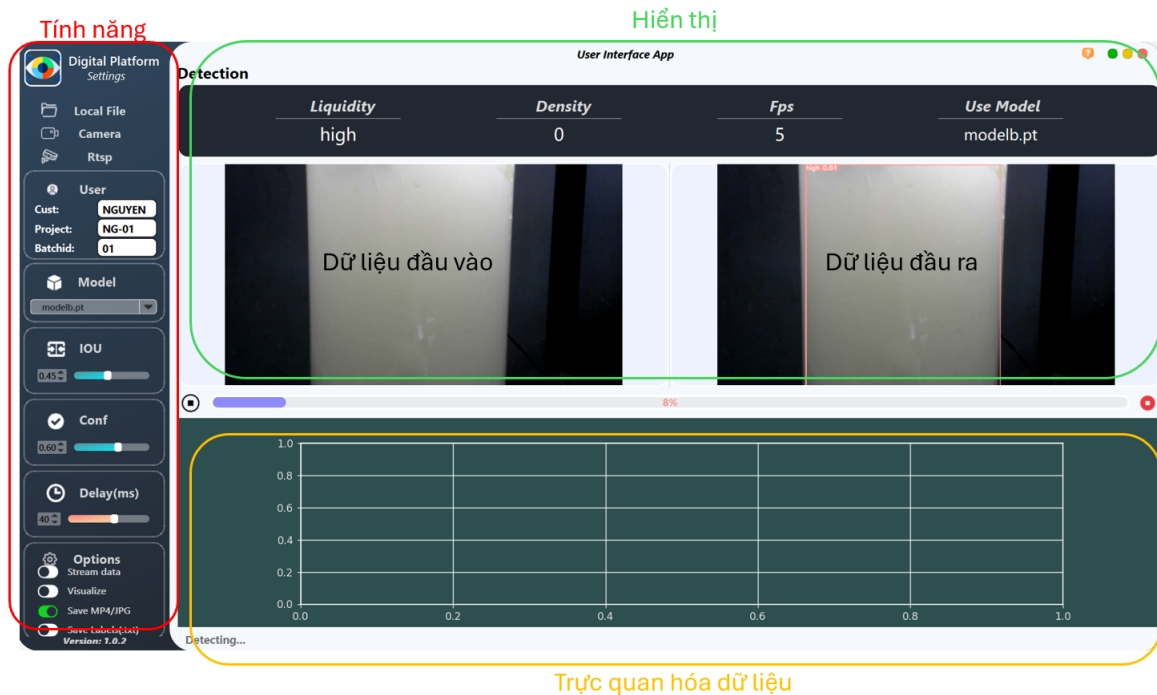


Hình 3.1: Thành phần hệ thống.

Bằng cách hiểu được kiến trúc và cách hoạt động của từng công nghệ, chúng ta có thể xây dựng được một hệ thống hiệu quả và đảm bảo hiệu suất cao. Tiếp theo, chúng tôi sẽ trình bày chi tiết về kiến trúc và nguyên lý hoạt động của từng công nghệ cụ thể.

3.1.1 Ứng dụng chạy mô hình Yolov8 thời gian thực

Trong ứng dụng này, chúng tôi thực hiện thiết kế giao diện và định nghĩa các tính năng cần thiết cho ứng dụng phát hiện bọt khí và trạng thái môi trường.



Hình 3.2: Ứng dụng Yolov8 thời gian thực.

Ứng dụng bao gồm các thành phần sau:

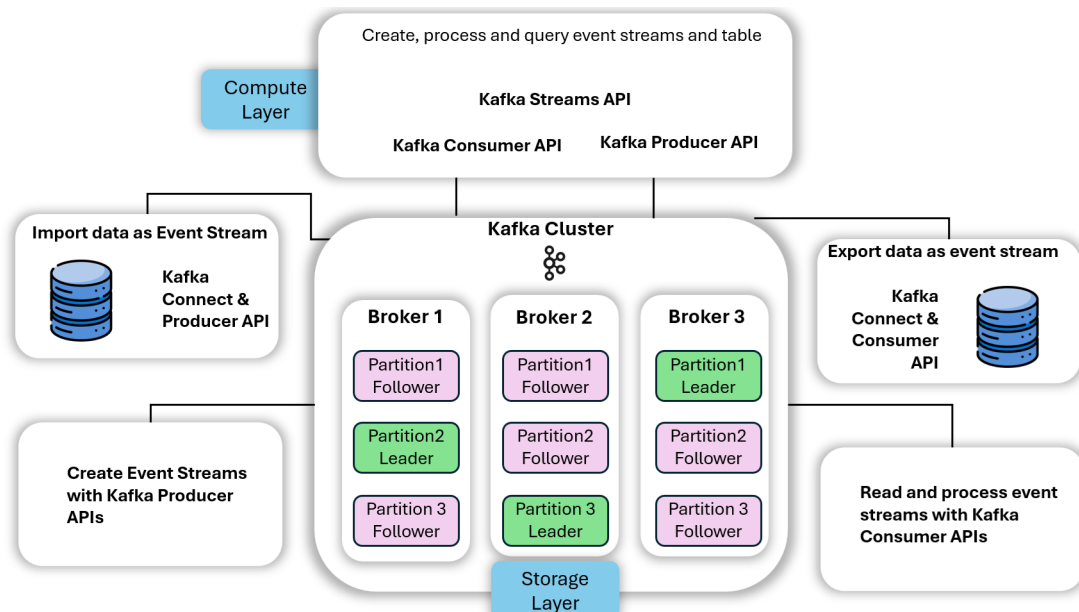
- **Tính năng:**
 - Nguồn dữ liệu: chúng tôi định nghĩa 3 nguồn dữ liệu bao gồm dữ liệu tĩnh là ảnh và video có sẵn, nguồn dữ liệu từ webcam hoặc điện thoại đã kết nối với máy tính và cuối cùng là nguồn dữ liệu từ camera từ xa.
 - Metadata: chúng tôi định nghĩa các thông tin cần phải cung cấp trước khi bắt đầu chạy ứng dụng bao gồm tên khách hàng, tên dự án và số ID của lô. Chúng tôi yêu cầu cung cấp thông tin nhằm mục đích quản lý và nhận diện khách hàng, dự án một cách dễ dàng và hiệu quả.
 - Các tùy chọn mô hình: chúng tôi thiết kế đơn giản giúp người dùng dễ dàng thao tác tùy vào mục đích và điều chỉnh các chỉ số giúp tăng độ chính xác mô hình như IoU, Conf, Delay.
 - Lựa chọn: chúng ta có thể chọn các lựa chọn tùy vào mong muốn. Bao gồm việc lưu dữ liệu kết quả, gửi dữ liệu trực tuyến lên hệ thống xử lý dữ liệu và hơn hết là trực quan hóa dữ liệu dự đoán ngay trên ứng dụng.
- **Hiển thị:** Chúng tôi thiết kế nhằm cung cấp cho người dùng thông tin về mô hình đang dự đoán trên nguồn dữ liệu về loại môi trường (yếu, bình thường và mạnh), số lượng bọt khí, tốc độ khung hình và tên mô hình đang sử dụng. Chúng tôi đưa ra hình ảnh đầu vào và kết quả đầu ra mô hình ngay trên ứng dụng, giúp người dùng theo dõi dễ dàng.
- **Trực quan hóa dữ liệu:** Vùng trực quan hóa dữ liệu giúp người dùng dễ dàng xem lượng phân bố của bọt khí theo thời gian thực ngay trên ứng dụng, giúp giám sát và kịp thời điều chỉnh nếu có vấn đề xảy ra.

3.1.2 Apache Kafka

Trong phần này, chúng ta sẽ cùng tìm hiểu về kiến trúc và nguyên lý hoạt động của Kafka. Tìm hiểu về cách brokers, Data Plane, Control Plane hoạt động, làm sao đảm bảo được những cam kết của một hệ thống dữ liệu lớn yêu cầu bao gồm độ bền, đảm bảo trình tự dữ liệu, tính chịu lỗi và tìm hiểu một số khả năng nâng cao hơn như tính linh hoạt, sao chép địa lý.

Kafka được thiết kế như hệ thống luồng sự kiện, nên cốt lõi có thể được xem là hệ thống lưu trữ. Hệ thống lưu trữ của kafka được thiết kế để lưu trữ sự kiện hiệu quả và cũng được thiết kế như hệ thống phân tán sao cho trong tương lai nhu cầu tăng có thể dễ dàng mở rộng để phục vụ nhu cầu tăng đó.

Kafka có hai API cơ bản dùng để truy cập dữ liệu được lưu trữ trong lớp lưu trữ. Một là Producer API cho phép tạo sự kiện và lưu trữ dữ liệu vào lớp lưu trữ. Hai là Consumer API cho phép ứng dụng đọc những sự kiện. Bên cạnh đó, Kafka còn có hai API cấp cao hơn. Đầu tiên là Connect API, được thiết kế để tích hợp Kafka với các hệ sinh thái khác, chẳng hạn như chúng ta có một nguồn dữ liệu bên ngoài và muốn đưa dữ liệu đó vào Kafka hoặc chúng ta cũng có thể đưa dữ liệu vào các hệ thống bên ngoài khác với Connect API. API cấp cao thứ hai được thiết kế cho việc xử lý, được gọi là Kafka Streams API. Chúng ta sẽ cùng nhau tìm hiểu chi tiết cấu tạo từng thành phần trong Kafka.

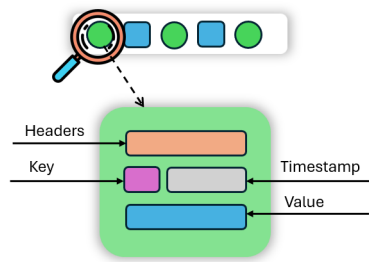


Hình 3.3: Kiến trúc tổng quát của Apache Kafka.

a) Sự kiện (Event)

Sự kiện được xem như là một khái niệm cốt lõi của Kafka. Sự kiện có thể là một đơn hàng, thanh toán, một lần nhấp vào một liên kết, do đó Event trong Kafka có thể xem là một bản ghi. Mỗi bản ghi bao gồm Header, Key, Timestamp, Value.

- Header chứa metadata về Event.
- Key là một thành phần quan trọng trong việc phân bổ vào các partitions trong Topic. Mặc định, Event có key giống nhau sẽ lưu vào cùng 1 partition trong Topic đó. Trường hợp key là giá trị rỗng (Null) thì các Event đó sẽ được lần lượt phân bổ vào các partition khác nhau.

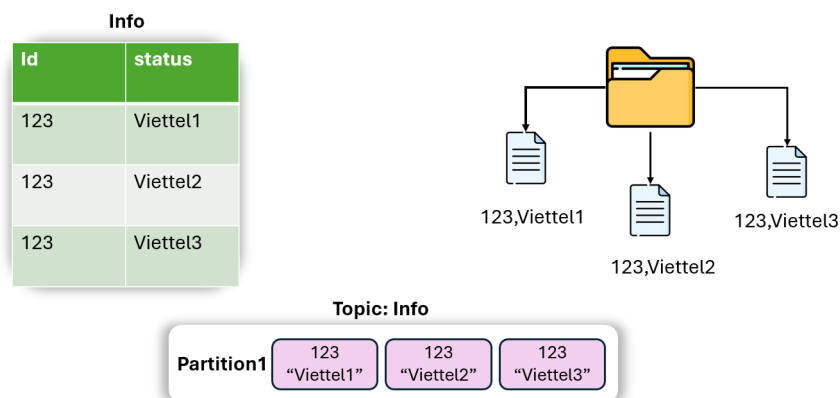


Hình 3.4: Cấu tạo của một sự kiện.

- Timestamp: Mỗi Event sẽ được kèm một mốc thời gian sinh ra sự kiện đó.
- Value chứa nội dung thực tế của bản ghi. Được mã hóa dưới dạng một byte array.

b) Chủ đề (Topic)

Trong Kafka, chủ đề có thể xem như là một cơ sở dữ liệu hay là một thư mục trong hệ thống tập tin. Mọi bản ghi trong Kafka sẽ được phân loại vào từng topic riêng dựa vào yêu cầu người dùng.



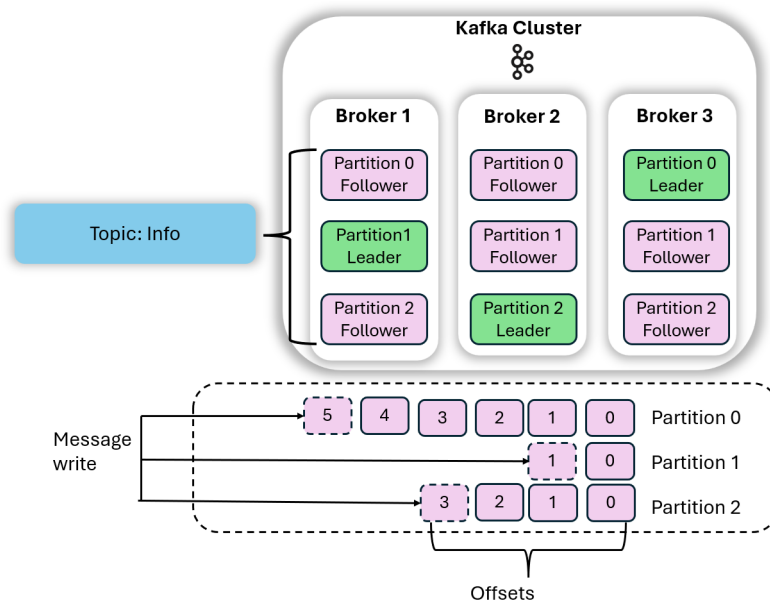
Hình 3.5: Tổng quan về chủ đề.

Một topic có thể đồng thời kết nối với nhiều Producer và Consumer cùng lúc. Mỗi Consumer khi đọc từ cùng 1 Topic sẽ có chỉ số offset riêng để lưu địa chỉ đọc trong các partition. Bản ghi trong Topic sẽ không xóa khi được tiêu thụ xong, thay vào đó các bản ghi sẽ được lưu theo yêu cầu của người dùng thiết lập, có thể cài đặt lưu trong 1 phút, 1 tuần hoặc lưu tới khi dung lượng đạt tới ngưỡng 7 GB.

• Partition

Mỗi chủ đề thường có nhiều phân vùng và các bản ghi được thêm vào cuối mỗi phân vùng. Phân vùng là cách mà Kafka cung cấp khả năng khôi phục dữ liệu và khả năng mở rộng. Mỗi phân vùng có thể lưu trữ trên một máy chủ khác nhau, vì vậy một chủ đề duy nhất có thể được thu nhỏ theo chiều ngang trên nhiều máy, cung cấp hiệu suất vượt xa khả năng của một máy chủ. Đề xuất trường hợp tốt nhất là 3 bản sao cho mỗi phân vùng trên cụm Kafka.

Một partition khi thực hiện sao chép trên nhiều Broker khác nhau sẽ được chia thành 2 loại: Leader và Follower (Hình 3.5). Partition Leader thực hiện nhiệm vụ đọc và ghi. Trong khi đó các Follower sẽ đảm nhiệm nhiệm vụ đồng bộ dữ liệu từ Partition Leader. Khi một broker bị dừng hoạt động, Kafka Controller sẽ thực hiện chọn bất kỳ một Partition Follower



Hình 3.6: Tính chịu lỗi của topic Info trên 3 brokers, 3 partitions và 3 replications.

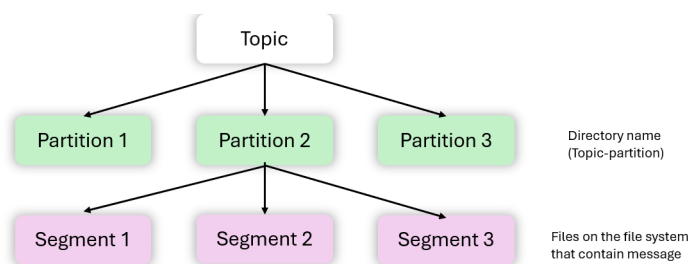
làm Partition Leader và Producer cùng với Consumer sẽ nối lại để thực hiện ghi và đọc dữ liệu.

- Partition offset

Mỗi sự kiện trong phân vùng chủ đề Kafka có một ID duy nhất gọi là offset. Đây là một số tăng dần đơn điệu và một khi nó được cấp, nó không bao giờ được sử dụng lại (hình 3.5). Vì vậy tất cả các sự kiện được lưu trữ trong chủ đề hoặc phân vùng cụ thể theo thứ tự offset đó, và nó sẽ được cấp và giao cho consumer theo thứ tự sự kiện đó, điều này sẽ làm cho việc theo dõi của consumer dễ dàng hơn.

- Segment

Các broker của Kafka chia mỗi phân vùng thành các đoạn (segment). Mỗi đoạn được lưu trữ trong một tệp dữ liệu duy nhất trên đĩa được gắn vào broker. Theo mặc định, mỗi đoạn chứa hoặc 1 GB dữ liệu hoặc dữ liệu trong một tuần, tùy thuộc vào giới hạn nào đạt trước. Khi broker của Kafka nhận dữ liệu cho một phân vùng, khi giới hạn đoạn được đạt tới, nó sẽ đóng tệp và bắt đầu một tệp mới.



Hình 3.7: Lưu trữ trên Kafka.

Chỉ có một đoạn (segment) là **HOẠT ĐỘNG** tại bất kỳ thời điểm nào - đó là đoạn dữ liệu đang được ghi vào. Một đoạn chỉ có thể bị xóa nếu nó đã được đóng trước đó. Kích thước của một đoạn được kiểm soát bởi hai cấu hình của Broker:

- log.segment.bytes: kích thước tối đa của một đoạn đơn lẻ tính bằng byte (mặc định là 1 GB)

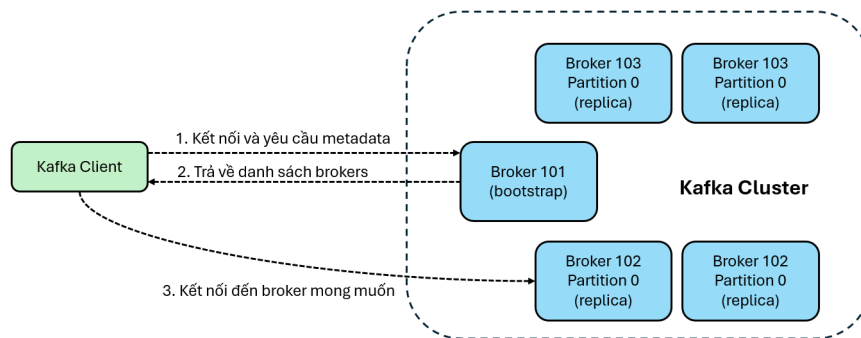
- log.segment.ms: thời gian Kafka sẽ chờ trước khi cam kết đoạn nếu chưa đầy (mặc định là 1 tuần)

c) Broker

Các broker của Kafka lưu trữ dữ liệu trong một thư mục trên đĩa máy chủ mà chúng chạy. Mỗi phân vùng chủ đề (topic-partition) nhận được một thư mục con riêng với tên liên quan đến chủ đề đó. Để đạt được thông lượng cao và khả năng mở rộng trên các chủ đề, các chủ đề Kafka được phân vùng. Nếu có nhiều broker Kafka trong một cụm, thì các phân vùng cho một chủ đề sẽ được phân phối đều giữa các broker để đạt được cân bằng tải và khả năng mở rộng (hình 3.5). Không có mối quan hệ giữa ID của broker và ID của phân vùng - Kafka làm rất tốt việc phân phối các phân vùng đều giữa các broker có sẵn. Trong trường hợp cụm trở nên mất cân bằng do quá tải của một broker cụ thể, các quản trị viên Kafka có thể cân bằng lại cụm và di chuyển các phân vùng.

Một client muốn gửi hoặc nhận tin nhắn từ Kafka Cluster có thể kết nối với bất kỳ broker nào trong cụm. Mỗi broker trong cụm có metadata về tất cả các broker khác và sẽ giúp client kết nối với chúng. Do đó, bất kỳ broker nào trong cụm cũng được gọi là bootstrap server.

Bootstrap server sẽ trả về metadata cho client, bao gồm danh sách tất cả các broker trong cụm. Sau đó, khi cần thiết, client sẽ biết chính xác broker nào để kết nối để gửi hoặc nhận dữ liệu, và xác định chính xác broker nào chứa phân vùng chủ đề liên quan.



Hình 3.8: Lưu trữ trên Kafka.

3.1.3 Telegraf

Telegraf là một chương trình dựa trên máy chủ để thu thập và gửi tất cả các số liệu và sự kiện từ cơ sở dữ liệu, hệ thống, và cảm biến IoT. Telegraf được viết bằng ngôn ngữ Go và biên dịch thành một tệp nhị phân duy nhất mà không có phụ thuộc bên ngoài và yêu cầu rất ít bộ nhớ. Các tính năng chính của Telegraf bao gồm:

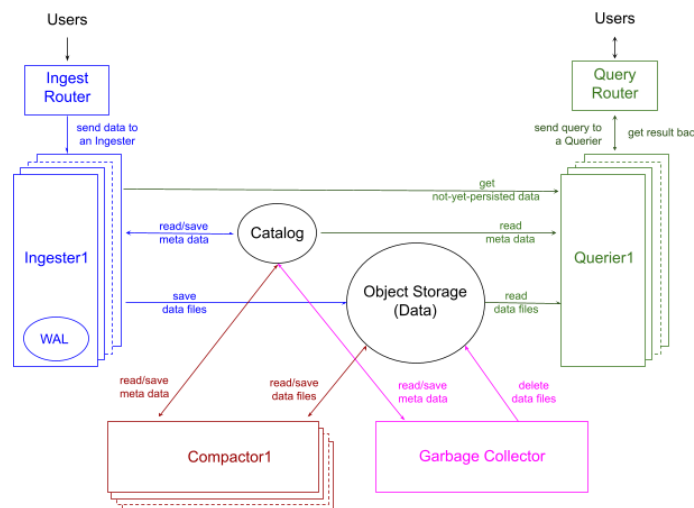
- Thu thập dữ liệu: Telegraf có khả năng thu thập dữ liệu từ nhiều nguồn khác nhau như hệ thống máy chủ, cơ sở dữ liệu, ứng dụng web, cảm biến IoT, v.v. Nó hỗ trợ nhiều giao thức như SNMP, HTTP, MQTT, và nhiều giao thức khác để thu thập dữ liệu từ các thiết bị và ứng dụng khác nhau.
- Gửi dữ liệu: Sau khi thu thập dữ liệu, Telegraf có thể gửi dữ liệu đến các nơi lưu trữ như InfluxDB, Elasticsearch, Kafka, và nhiều nơi lưu trữ khác.

- Cấu hình linh hoạt: Telegraf cung cấp cách thức cấu hình linh hoạt cho người dùng để thiết lập các plugin thu thập dữ liệu theo nhu cầu của họ. Người dùng có thể chọn và cấu hình các plugin để thu thập dữ liệu từ các nguồn cụ thể mà họ quan tâm.
- Độ tin cậy và hiệu suất: Telegraf được thiết kế để có độ tin cậy cao và hiệu suất tối ưu, với khả năng xử lý và gửi dữ liệu một cách hiệu quả ngay cả khi đối mặt với lưu lượng cao.
- Hỗ trợ cộng đồng và mở rộng: Vì là mã nguồn mở, Telegraf không ngừng phát triển và được mở rộng để hỗ trợ các tính năng và giao thức mới, đáp ứng nhu cầu người dùng trong việc thu thập và quản lý dữ liệu.

Telegraf là một công cụ mạnh mẽ và linh hoạt trong việc thu thập dữ liệu và là một phần quan trọng của hệ sinh thái giám sát và phân tích dữ liệu hiện đại.

3.1.4 InfluxDB

InfluxDB là một cơ sở dữ liệu chuỗi thời gian mã nguồn mở. InfluxDB bao gồm các API để lưu trữ và truy vấn dữ liệu, xử lý dữ liệu trong nền cho các mục đích ETL (Extract-Transform-Load), giám sát và cảnh báo, điều khiển, trực quan hóa và khám phá dữ liệu, và nhiều hơn nữa.



Hình 3.9: Kiến trúc của InfluxDB.

- Thành phần tiếp nhận dữ liệu xử lý quá trình nhập dữ liệu vào cơ sở dữ liệu. Người dùng ghi dữ liệu vào Ingest Router, sau đó dữ liệu được phân chia tới một trong các Ingesters, cho phép số lượng ingester được mở rộng tùy theo khối lượng công việc dữ liệu. Mỗi ingester nhận dạng bảng, xác nhận schema dữ liệu, phân vùng dữ liệu theo ngày trên cột "time", loại bỏ dữ liệu trùng lặp, và lưu trữ nó dưới dạng tệp Parquet. Ingesters cũng cập nhật Catalog về tệp mới được tạo, báo hiệu cho các thành phần khác rằng dữ liệu mới đã đến. InfluxDB tối ưu hóa đường dẫn ghi để giữ độ trễ ghi ở mức tối thiểu, trong khoảng vài mili giây.
- Thành phần truy vấn dữ liệu xử lý các truy vấn của người dùng bằng SQL hoặc InfluxQL. Người dùng gửi truy vấn đến Query Router, sau đó chuyển tiếp chúng đến một Querier. Querier đọc dữ liệu cần thiết, xây dựng kế hoạch cho truy vấn, thực thi nó, và trả kết quả. Queriers có thể mở rộng tùy theo khối lượng công việc truy vấn. Chúng thực hiện các nhiệm vụ như lưu trữ metadata, đọc và lưu trữ dữ liệu, giao tiếp với ingesters cho dữ liệu chưa được lưu trữ, và xây dựng và thực thi một kế hoạch truy vấn tối ưu.

- Nén dữ liệu là một quá trình giải quyết thách thức của việc có nhiều tệp nhỏ được lưu trữ trong Object Storage, điều này có thể làm giảm hiệu suất truy vấn. Compactors chạy trong các công việc nền để đọc các tệp mới nhập và nén chúng thành ít tệp hơn, lớn hơn, và không trùng lặp. Quá trình này cải thiện hiệu suất truy vấn bằng cách giảm đáng kể I/O trong thời gian truy vấn.
Số lượng compactors có thể được mở rộng dựa trên khối lượng công việc nén, xem xét các yếu tố như số lượng bảng với các tệp dữ liệu mới, kích thước của các tệp, và số lượng tệp hiện có mà các tệp mới chồng lấn.
- Thành phần thu gom rác quản lý việc giữ lại dữ liệu và thu hồi không gian trong cơ sở dữ liệu. Nó hoạt động thông qua các công việc nền mà lập lịch cả việc xóa dữ liệu mềm và cứng.
- Catalog bao gồm metadata như thông tin về cơ sở dữ liệu, bảng, cột, và tệp, và InfluxDB sử dụng cơ sở dữ liệu tương thích với Postgres để quản lý Catalog này. Mặt khác, Object Storage chỉ chứa các tệp Parquet và có thể được lưu trữ trên các nền tảng như ổ đĩa cục bộ, Amazon S3, Azure Blob Storage, hoặc Google Cloud Storage.

3.1.5 Grafana

Grafana là một ứng dụng web mã nguồn mở được sử dụng để trực quan hóa dữ liệu. Được sử dụng làm công cụ giám sát và cảnh báo dữ liệu. Grafana cho phép chúng ta tạo biểu đồ, đồ thị và bảng dữ liệu mà chúng ta muốn trực quan hóa. Nó sử dụng hệ thống truy vấn để xây dựng các hiển thị tùy chỉnh từ bộ dữ liệu của chúng ta. Ngoài ra, nó cho phép đặt cảnh báo, sẽ kích hoạt khi đạt điều kiện cụ thể. Hơn nữa, Grafana giám sát dữ liệu của chúng ta theo thời gian thực và cho phép giám sát.

Các tính năng chính của Grafana bao gồm:

- Giám sát: Grafana cho phép người dùng giám sát các khía cạnh khác nhau của nguồn dữ liệu. Tùy thuộc vào nguồn dữ liệu và yêu cầu của chúng ta, có thể là từ sử dụng CPU đến thời gian tra cứu.
- Cảnh báo: Xây dựng trên tính năng giám sát, Grafana cho phép chúng ta thiết lập công cụ cảnh báo. Điều này cho phép chúng ta nhận thông báo qua email, cửa sổ pop-up hoặc các công cụ bên thứ ba khi một điều kiện cụ thể được thực hiện.
- Bảng điều khiển: Các bảng điều khiển được điền thông tin số liệu và nhật ký được kết hợp để tạo thành một giao diện bảng điều khiển. Các bảng này có thể được chỉnh sửa hoàn toàn và cho phép tùy chỉnh từ việc thay đổi tên đến thay đổi loại biểu diễn dữ liệu.
- Hành động: Các logic điều kiện có thể được áp dụng cho các bảng điều khiển này, cho phép trải nghiệm linh hoạt. Ví dụ: khi giá trị của một báo cáo số liệu nằm trong một phạm vi cụ thể, đầu ra trên bảng điều khiển thay đổi màu sắc sang màu đỏ để chỉ ra lỗi.
- Plugin: Grafana cũng cho phép bảng điều khiển truy cập vào các API bên thứ ba, điều này nâng cao trải nghiệm chúng ta, tạo ra tính khả dụng cao hơn.

3.2 Xây dựng hệ thống phát hiện bọt khí thời gian thực

Chúng tôi sẽ trình bày các bước tổng quát để xây dựng hệ thống phát hiện bọt khí thời gian thực. Bằng cách này, chúng ta sẽ hiểu được cách triển khai hệ thống và hệ thống thật sự hoạt động.

Bước 1: Chuẩn bị Môi trường

- **Tạo Máy ảo trên Azure Cloud:**
 - Đăng nhập vào Azure Portal.
 - Tạo một máy ảo với cấu hình phù hợp cho hệ thống của bạn.
 - Thiết lập mạng và bảo mật cần thiết.
- **Cài đặt Docker và Docker Compose trên Máy ảo:**
 - Cập nhật hệ thống và cài đặt Docker.
 - Cài đặt Docker Compose để quản lý các container.

Bước 2: Cấu hình Docker Compose

- **Tạo file `docker-compose.yml`:**
 - Định nghĩa các dịch vụ Kafka, Zookeeper, Telegraf, InfluxDB và Grafana.
 - Thiết lập các kết nối mạng và volume cần thiết.

Bước 3: Cấu hình Telegraf

- **Tạo file cấu hình `telegraf.conf`:**
 - Cấu hình Telegraf để lấy dữ liệu từ Kafka và gửi đến InfluxDB.
 - Định nghĩa các input plugin cho Kafka và output plugin cho InfluxDB.

Bước 4: Cấu hình GitLab CI/CD

- **Tạo file `.gitlab-ci.yml`:**
 - Định nghĩa các stages cho CI/CD pipeline, bao gồm build và deploy.
 - Cấu hình các job để build Docker images và triển khai các dịch vụ.

Bước 5: Triển khai trên Máy ảo Azure Cloud

- **Chuẩn bị Repository trên GitLab:**
 - Tạo một repository mới trên GitLab.
 - Đẩy mã nguồn và các file cấu hình lên repository.
- **Thiết lập CI/CD Pipeline trên GitLab:**
 - Cấu hình GitLab Runner để chạy các job trong pipeline.
 - Kích hoạt pipeline để tự động build và deploy các dịch vụ.
- **Triển khai Dịch vụ:**
 - Sử dụng Docker Compose để khởi động các dịch vụ trên máy ảo Azure.
 - Kiểm tra các dịch vụ để đảm bảo rằng chúng đang chạy và kết nối đúng cách.

Trên đây là các bước xây dựng hệ thống phát hiện bọt khí thời gian thực dựa vào các công nghệ đã được trình bày ở trên. Chúng tôi đã sử dụng mô hình Yolov8 để phát hiện bọt khí trong nuôi vi tảo, vì vậy chúng ta sẽ tìm hiểu về các kiến thức cần biết trong thị giác máy tính, kiến trúc và cách hoạt động của mô hình Yolov8 để huấn luyện mô hình hiệu quả và chính xác.

3.3 Kiến thức nền tảng trong thị giác máy tính

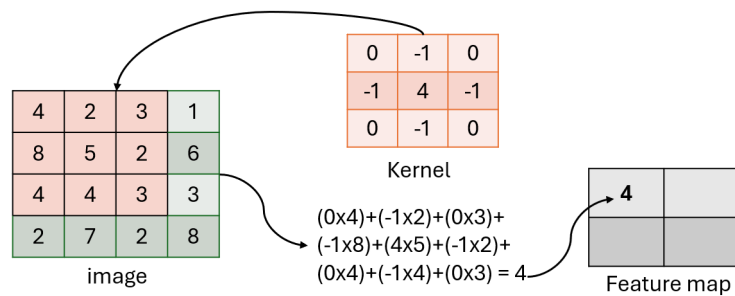
Trong thị giác máy tính và học sâu, việc nắm vững các khái niệm cơ bản như tích chập (Conv), bộ lọc (kernel), bước nhảy (stride) và đệm biên (padding) là nền tảng để xây dựng các mô hình hiệu quả và tối ưu. Chúng ta sẽ tìm hiểu các kiến thức nền tảng trước khi đi vào tìm hiểu kiến trúc YOLOv8 cụ thể.

3.3.1 Tích chập (Conv)

Kiến trúc YOLO áp dụng phương pháp phân tích đặc trưng cục bộ thay vì xem xét toàn bộ hình ảnh, mục tiêu chính là giảm nỗ lực tính toán và cho phép phát hiện đối tượng trong thời gian thực. Để trích xuất các bản đồ đặc trưng, các phép tích chập được sử dụng nhiều lần trong thuật toán.

Tích chập là một phép toán toán học kết hợp hai hàm để tạo ra một hàm thứ ba. Trong thị giác máy tính và xử lý tín hiệu, tích chập thường được sử dụng để áp dụng các bộ lọc lên hình ảnh hoặc tín hiệu, làm nổi bật các mẫu cụ thể. Trong các mạng nơ-ron tích chập (CNNs), tích chập được sử dụng để trích xuất các đặc trưng từ các đầu vào như hình ảnh. Các phép tích chập được cấu trúc bởi các Kernel (K), Stride (s) và Padding (p).

3.3.2 Bộ lọc (Kernel)



Hình 3.10: Ví dụ về kernel 3x3.

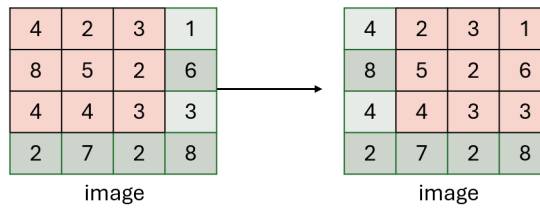
Kernel, còn được gọi là bộ lọc, là một mảng bao gồm các số được trượt qua đầu vào (hình ảnh hoặc tín hiệu) trong quá trình tích chập. Mục tiêu là áp dụng các phép toán cục bộ lên đầu vào để phát hiện các đặc điểm cụ thể. Mỗi phần tử trong kernel đại diện cho một trọng số được nhân với giá trị tương ứng trong đầu vào trong quá trình tích chập.

3.3.3 Bước nhảy (Stride)

Stride là số pixel thay đổi trên ma trận đầu vào. Khi stride là 1 thì ta di chuyển các kernel 1 pixel. Khi stride là 2 thì ta di chuyển các kernel đi 2 pixel và tiếp tục như vậy.

3.3.4 Đệm biên (Padding)

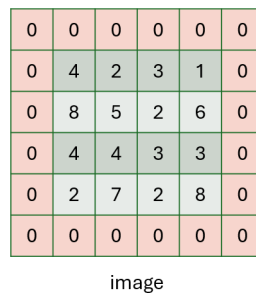
"Padding" là việc thêm các điểm ảnh bổ sung xung quanh các cạnh của hình ảnh đầu vào (thường là các số 0) trước khi áp dụng các phép toán tích chập. Việc này nhằm đảm bảo rằng



Hình 3.11: Ví dụ về stride=1.

thông tin tại các cạnh của hình ảnh được xử lý giống như thông tin ở trung tâm trong các phép toán tích chập.

Khi một bộ lọc (kernel) được áp dụng lên một hình ảnh, nó thường đi qua từng điểm ảnh của hình ảnh. Nếu không áp dụng padding, các điểm ảnh ở cạnh của hình ảnh sẽ có ít điểm lân cận hơn so với các điểm ảnh ở trung tâm, điều này có thể dẫn đến tổn thất thông tin ở các vùng này.

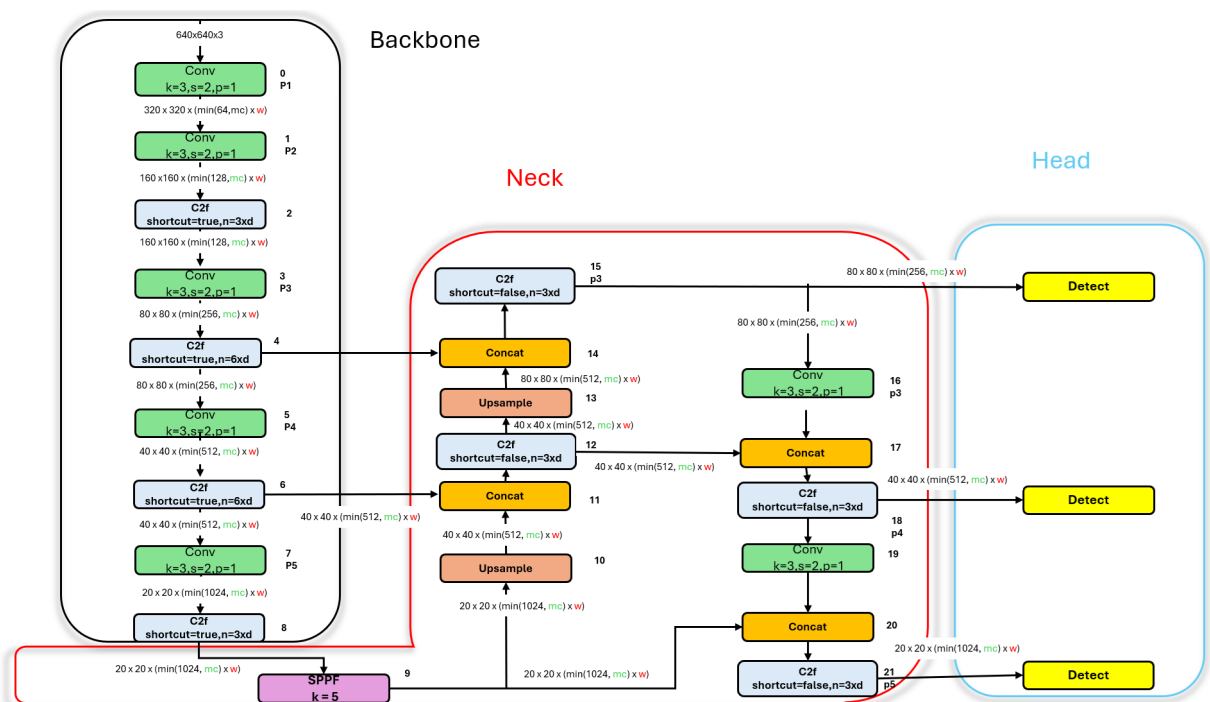


Hình 3.12: Ví dụ về padding 0.

3.4 Mô hình YOLOv8

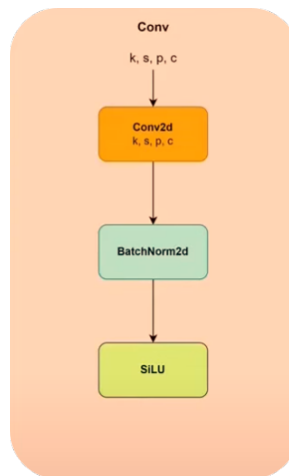
3.4.1 Kiến trúc

Bước đầu tiên để hiểu kiến trúc YOLO là hiểu 3 khối quan trọng trong thuật toán và mọi thứ sẽ diễn ra trong các khối này, đó là: Backbone (Xương sống), Neck (Cổ) và Head (Đầu). Chức năng của mỗi khối được mô tả dưới đây.



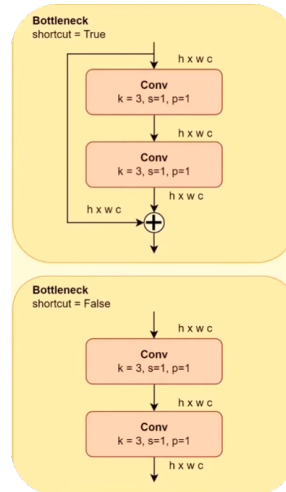
Hình 3.13: Kiến trúc YOLOv8.

* Conv



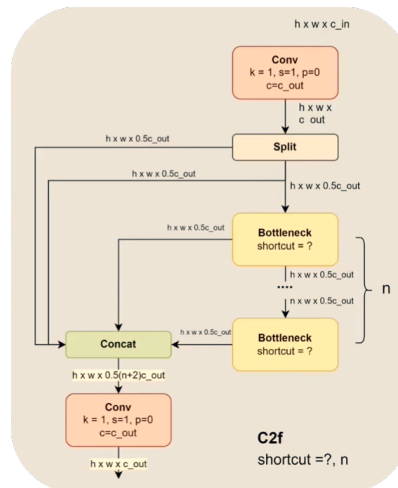
Hình 3.14: Cấu trúc lớp conv.

* Bottleneck



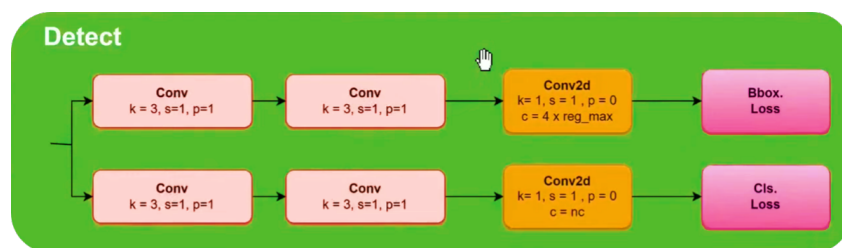
Hình 3.15: Cấu trúc lớp Bottleneck.

* C2f



Hình 3.16: Cấu trúc lớp C2f.

* Detect



Hình 3.17: Cấu trúc Detect.

• Backbone (Xương sống):

- Chức năng: Backbone, còn được gọi là bộ trích xuất đặc trưng, có nhiệm vụ trích xuất các đặc trưng có ý nghĩa từ đầu vào.

- Hoạt động: Nắm bắt các mẫu đơn giản trong các lớp ban đầu, chẳng hạn như cạnh và kết cấu. Có thể có nhiều thang đo biểu diễn khi tiến hành, nắm bắt các đặc trưng từ các mức độ trừu tượng khác nhau. Sẽ cung cấp một biểu diễn phong phú và phân cấp của đầu vào.
- Neck (Cổ):
 - Chức năng: Neck hoạt động như một cầu nối giữa Backbone và Head, thực hiện các thao tác hợp nhất đặc trưng và tích hợp thông tin ngữ cảnh. Về cơ bản, Neck lắp ráp các đặc trưng bằng cách tập hợp các bản đồ đặc trưng thu được từ Backbone.
 - Hoạt động: Thực hiện kết hợp hoặc hợp nhất các đặc trưng ở các thang đo khác nhau để đảm bảo rằng mạng có thể phát hiện các đối tượng với kích thước khác nhau. Tích hợp thông tin ngữ cảnh để cải thiện độ chính xác của việc phát hiện bằng cách xem xét ngữ cảnh rộng hơn. Giảm độ phân giải không gian và kích thước của tài nguyên để tạo điều kiện thuận lợi cho việc tính toán.
- Head (Đầu):
 - Chức năng: Head là phần cuối cùng của mạng và chịu trách nhiệm tạo ra các đầu ra của mạng, chẳng hạn như các hộp giới hạn (bounding boxes) và điểm tin cậy cho việc phát hiện đối tượng.
 - Hoạt động: Tạo ra các hộp giới hạn liên kết với các đối tượng có thể có trong hình ảnh. Gán điểm tin cậy cho mỗi hộp giới hạn để chỉ ra khả năng có mặt của một đối tượng. Phân loại các đối tượng trong các hộp giới hạn theo danh mục của chúng.

3.4.2 Tổn thất trong phát hiện đối tượng

Trong huấn luyện mô hình phát hiện đối tượng, các hàm tổn thất đóng một vai trò quan trọng. Các hàm tổn thất định lượng sự sai khác giữa các hộp giới hạn được dự đoán và các chú thích một hộp giới hạn thực tế (ground truth), cung cấp một đánh giá về việc mô hình học tập trong quá trình huấn luyện. Các thành phần tổn thất phổ biến trong phát hiện đối tượng bao gồm:

- **Box loss (box_loss)**
Box loss đo lường sai số trong việc dự đoán các tọa độ của hộp giới hạn. Nó thúc đẩy mô hình điều chỉnh các hộp giới hạn dự đoán để phù hợp với các hộp giới hạn ground truth.
- **Class loss (cls_loss)**
Class loss định lượng sai số trong việc dự đoán lớp đối tượng cho mỗi hộp giới hạn. Nó đảm bảo rằng mô hình xác định chính xác danh mục của đối tượng.
- **Defocus loss (dfl_loss)**
Defocus loss là một thành phần tổn thất chuyên biệt giúp cải thiện phát hiện đối tượng trong các tình huống ảnh bị mờ hoặc không rõ nét. Nó khuyến khích mô hình tập trung vào cải thiện việc phát hiện trong những điều kiện thách thức.

3.4.3 Hàm tổn thất và quy tắc cập nhật trọng số

Hàm tổn thất tổng quát và quy trình cập nhật trọng số có thể được định nghĩa như sau:

$$L(\theta) = \frac{\lambda_{box}}{N_{pos}} L_{box}(\theta) + \frac{\lambda_{cls}}{N_{pos}} L_{cls}(\theta) + \frac{\lambda_{dfl}}{N_{pos}} L_{dfl}(\theta) + \varphi ||\theta||_2^2. \quad (3.1)$$

$$V_t = \beta V^{t-1} + \nabla_{\theta} L(\theta^{t-1}) \quad (3.2).$$

$$\theta_t = \theta^{t-1} - \eta V^t (3.3).$$

(3.1) là hàm tổn thất tổng quát kết hợp các trọng số tổn thất, (3.2) là thuật ngữ vận tốc với động lượng β , và (3.3) là quy tắc cập nhật trọng số với η là tốc độ học tập. Hàm tổn thất cụ thể của Yolov8 có thể được định nghĩa như sau:

$$\begin{aligned} \mathcal{L} = & \frac{\lambda_{box}}{N_{pos}} \sum_{x,y} \mathbb{1}_{c_{x,y}^*} \left[1 - q_{x,y} + \frac{\|b_{x,y} - \hat{b}_{x,y}\|_2^2}{\rho^2} + \alpha_{x,y} \nu_{x,y} \right] \\ & + \frac{\lambda_{cls}}{N_{pos}} \sum_{x,y} \sum_{c \in classes} y_c \log(\hat{y}_c) + (1 - y_c) \log(1 - \hat{y}_c) \\ & + \frac{\lambda_{dfl}}{N_{pos}} \sum_{x,y} \mathbb{1}_{c_{x,y}^*} \left[- (q_{(x,y)+1} - q_{x,y}) \log(\hat{q}_{x,y}) \right. \\ & \quad \left. + (q_{x,y} - q_{(x,y)-1}) \log(\hat{q}_{(x,y)+1}) \right] \end{aligned}$$

Trong đó:

$$q_{x,y} = IoU_{x,y} = \frac{\beta_{x,y} \cap \hat{\beta}_{x,y}}{\beta_{x,y} \cup \hat{\beta}_{x,y}} \quad (3.4)$$

$$\nu_{x,y} = \frac{4}{\pi^2} \left(\arctan \left(\frac{w_{x,y}}{h_{x,y}} \right) - \arctan \left(\frac{\hat{w}_{x,y}}{\hat{h}_{x,y}} \right) \right)^2 \quad (3.5)$$

$$\alpha_{x,y} = \frac{\nu_{x,y}}{1 - q_{x,y}} \quad (3.6)$$

$$\hat{y}_c = \sigma(\cdot) \quad (3.7)$$

$$\hat{q}_{x,y} = \text{softmax}(\cdot) \quad (3.8)$$

Và:

- N_{pos} là tổng số ô chứa một đối tượng.
- $\mathbf{1}_{x,y}^c$ là một hàm chỉ báo cho các ô chứa một đối tượng.
- $\beta_{x,y}$ là một tuple đại diện cho hộp bao gồm (xcoord, ycoord, width, height).
- $\hat{\beta}_{x,y}$ là hộp dự đoán tương ứng của ô đó.
- $b_{x,y}$ là một tuple đại diện cho điểm trung tâm của hộp chứa.
- y_c là nhãn thực cho lớp c (không phải ô lưới c) cho mỗi ô lưới riêng lẻ (x, y) trong đầu vào, bất kể có đối tượng hay không.
- $q_{(x,y) \pm 1}$ là các hộp dự đoán IoU gần nhất (trái và phải) $\in \mathbf{1}_{x,y}^c$.
- $w_{x,y}$ và $h_{x,y}$ là chiều rộng và chiều cao tương ứng của các hộp.
- ρ là độ dài đường chéo của hộp chứa nhỏ nhất bao phủ các hộp dự đoán và hộp thực.

Mỗi ô sau đó xác định ứng cử viên tốt nhất để dự đoán hộp chứa của đối tượng. Hàm tổn thất này bao gồm tổn thất IoU hoàn chỉnh (CIoU) được đề xuất bởi Zheng *et al.* [12] như là tổn thất của hộp, entropy chéo nhị phân chuẩn cho phân loại đa nhãn như là tổn thất phân loại (cho phép mỗi ô dự đoán nhiều hơn một lớp), và tổn thất tiêu điểm phân phối được đề xuất bởi Li *et al.* [13].

Chương 4

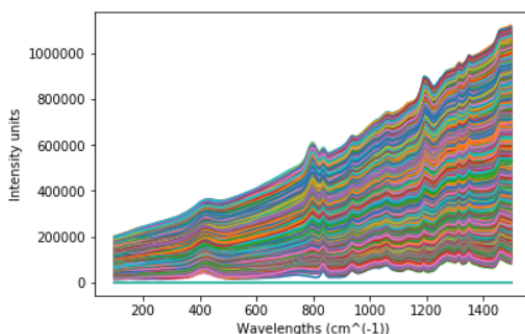
Thực nghiệm và đánh giá

Chúng tôi thực hiện áp dụng các bộ dữ liệu vào hệ thống và đánh giá các thí nghiệm thông qua các thang đo. Trong chương này, chúng tôi sẽ trình bày các thực nghiệm đã thực hiện và đánh giá cụ thể chúng. Đối với hệ thống truyền phát dữ liệu Raman thời gian thực, chúng tôi dùng bộ dữ liệu phổ Raman và hệ thống nhận diện bọt khí trong nuôi vi tảo, chúng tôi thực hiện cài đặt và mô phỏng lại quá trình nuôi vi tảo trong công nghiệp để thu thập dữ liệu về ảnh bọt khí.

4.1 Bộ dữ liệu phổ Raman

Bộ dữ liệu được cung cấp từ tiến sĩ hướng dẫn luận văn hiện đang sinh sống và làm việc ở Na Uy. Bộ dữ liệu bao gồm 100 lô sản xuất tổng cộng khoảng 2.5 GB dữ liệu, với dữ liệu phép đo phổ Raman để hỗ trợ nghiên cứu và phát triển trong lĩnh vực phân tích dữ liệu lớn, học máy và trí tuệ nhân tạo, đặc biệt là trong nuôi vi tảo trong công nghiệp nhằm mục đích dự đoán nồng độ hóa chất AAA tạo ra trong quá trình nuôi vi tảo. Bộ dữ liệu được thu thập bằng thiết bị phổ Raman Kaiser 1000 RXN, sử dụng cảm biến indium gallium arsenide (InGaAs) với phạm vi phổ từ 200-2400 cm^{-1} và độ phân giải 3 cm^{-1} . Thiết bị phân tích phổ Raman được cài đặt để ghi lại một phổ mỗi phút. Tổng cộng 540 phổ được ghi lại trong suốt 9 giờ.

Dữ liệu phổ Raman được sử dụng để theo dõi và phân tích các thành phần hóa học trong quá trình nuôi vi tảo. Điều này bao gồm việc đo lường nồng độ của chất AAA và các sản phẩm phụ khác trong quá trình trao đổi chất. Thiết bị Raman thu lại tín hiệu của sự thay đổi nồng độ các sản phẩm liên tục, và mô hình machine learning giúp cho việc chuyển đổi các tín hiệu này thành nồng độ sản phẩm theo thời gian thực, điều này giúp cho việc phân tích và điều chỉnh các điều kiện để tối ưu hóa sản xuất chất AAA.



Hình 4.1: Biểu đồ về bộ dữ liệu Raman.

4.2 Bộ dữ liệu bọt khí trong nuôi vi tảo

Với bộ dữ liệu bọt khí, chúng tôi thực hiện tự thu thập dữ liệu bằng cách tự lắp đặt thiết bị mô phỏng lại quá trình sục khí CO₂ trong nuôi vi tảo trong công nghiệp. Sau đó, chúng tôi thực hiện thu thập dữ liệu bọt khí dựa trên bộ thiết bị mô phỏng này. Chúng tôi tiến hành cài đặt và kết hợp linh hoạt các van trên ống dẫn thổi khí để tạo ra tốc độ sục khí riêng biệt cho từng loại trạng thái gần với các trường hợp có thể xảy ra trong nuôi vi tảo ở thực tế, với sự hỗ trợ của Tiến sĩ hỗ trợ luận văn hiện đang sinh sống và làm việc tại Nauy, là một chuyên gia trong lĩnh vực này.

4.2.1 Cài đặt thiết bị

Để thực hiện mô phỏng quá trình nuôi vi tảo trong công nghiệp và thu thập dữ liệu bọt khí để huấn luyện mô hình Yolov8, chúng tôi đã thực hiện mua và cài đặt thiết bị. Thành phần của bộ thiết bị mô phỏng bao gồm:

- Ống thủy tinh trong suốt hình trụ: kích thước 10x40.
- Máy bơm: công suất 3.5w.
- Ống nhựa và các van dùng để điều chỉnh tốc độ.



Hình 4.2: Cài đặt thiết bị mô phỏng nuôi vi tảo.

4.2.2 Mô tả bộ dữ liệu

Bộ dữ liệu này được thu thập nhằm phục vụ nghiên cứu và phân tích trong lĩnh vực nuôi vi tảo, tập trung vào việc nhận diện và đánh giá bọt khí xuất hiện trong quá trình nuôi. Đặc biệt, các thiết bị và quy trình thu thập dữ liệu được chúng tôi tự thiết lập nhằm mô phỏng một cách chính xác môi trường nuôi vi tảo trong công nghiệp.

Bộ dữ liệu hơn 700 ảnh chụp bọt khí với kích thước 640x640 chia đều ở 3 trạng thái: yếu, bình thường và mạnh. Bằng cách này, chúng tôi đã mô phỏng lại quá trình sục khí CO₂ khi nuôi vi tảo trong công nghiệp hiện nay và thực hiện các nghiên cứu về dữ liệu lớn và các mô hình học sâu.



Hình 4.3: Ví dụ 3 trạng thái trong bộ dữ liệu.

Sau khi thực nghiệm huấn luyện mô hình máy học và học sâu trên các bộ dữ liệu, chúng ta cần đánh giá độ chính xác của các mô hình và xem xét liệu mô hình đã đáp ứng yêu cầu để có thể triển khai trong thực tế. Để thực hiện việc này, chúng ta cần phân tích và đánh giá các chỉ số đánh giá mô hình tương ứng. Tiếp theo, chúng tôi sẽ trình bày các khái niệm và công thức toán học của các chỉ số sử dụng để đánh giá 2 mô hình sử dụng trong bài.

4.3 Chỉ số đánh giá mô hình

Bằng cách nắm rõ công thức toán học và ý nghĩa của từng chỉ số đánh giá mô hình, chúng ta có thể dễ dàng đánh giá mô hình huấn luyện một cách hiệu quả và chính xác. Chúng tôi sẽ trình bày khái niệm của từng chỉ số đánh giá mô hình và cách hoạt động của chúng.

4.3.1 Sai số trung bình tuyệt đối (MAE)

Mean Absolute Error (MAE) là trung bình của độ chênh lệch giữa các giá trị ban đầu và giá trị dự đoán. MAE cung cấp cho chúng ta độ đo về mức độ sai lệch giữa các dự đoán so với kết quả thực tế. Tuy nhiên, MAE không cung cấp cho chúng ta bất kỳ ý tưởng nào về hướng của sai số, tức là liệu chúng ta đang dự đoán thiếu hay dự đoán quá mức dữ liệu. Công thức của MAE như sau:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Trong đó:

- y_i là giá trị thực tế.
- $\hat{y}_i$ là giá trị dự đoán của mô hình.
- n là số lượng quan sát.

4.3.2 Trung bình bình phương sai số (MSE)

Mean Squared Error (MSE) tương đối giống với Mean Absolute Error (MAE), với điểm khác biệt duy nhất là MSE tính trung bình của bình phương của sai số giữa giá trị ban đầu y_i và giá trị dự đoán $\hat{y}_i$. Lợi ích của MSE là để tính toán độ dốc, trong khi MAE yêu cầu các công cụ lập trình tuyến tính phức tạp để tính toán độ dốc. Bằng cách lấy bình phương của sai số, các sai

số lớn sẽ trở nên rõ rệt hơn so với các sai số nhỏ, do đó mô hình có thể tập trung nhiều hơn vào các sai số lớn hơn. Công thức của MSE như sau:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Trong đó:

- y_i là giá trị thực tế.
- $\hat{y}_i$ là giá trị dự đoán của mô hình.
- n là số lượng quan sát.

4.3.3 R^2

Chỉ số R^2 giúp chúng ta đánh giá mức độ phù hợp của mô hình so với một mô hình đơn giản sử dụng giá trị trung bình của dữ liệu làm dự đoán. Một giá trị R^2 cao chỉ ra rằng mô hình dự đoán tốt hơn mô hình cơ bản, trong khi một giá trị R^2 thấp cho thấy mô hình có thể không dự đoán tốt hơn so với việc chỉ sử dụng giá trị trung bình.

Công thức của R^2 như sau:

$$R^2 = 1 - \frac{MSE(\text{model})}{MSE(\text{baseline})}.$$

- $MSE(\text{model})$ là Mean Squared Error của các dự đoán so với giá trị thực tế:

$$MSE_{\text{model}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

- $MSE(\text{baseline})$ là Mean Squared Error của giá trị trung bình so với giá trị thực tế:

$$MSE_{\text{baseline}} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2.$$

Trong đó:

- y_i là giá trị thực tế
- $\hat{y}_i$ là giá trị dự đoán của mô hình
- $\bar{y}$ là giá trị trung bình của các giá trị thực tế
- n là số lượng quan sát

4.3.4 Độ bao phủ (Recall)

Recall là một chỉ số được sử dụng trong các bài toán phân loại để đánh giá khả năng của mô hình trong việc nhận diện chính xác các mẫu dương tính. Toán học, recall được định nghĩa như sau:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Trong đó:

- TP (True Positives) là số lượng mẫu dương tính được dự đoán đúng.
- FN (False Negatives) là số lượng mẫu dương tính bị dự đoán sai thành âm tính.

Chỉ số recall cho biết tỷ lệ các mẫu dương tính được mô hình nhận diện chính xác trong tổng số các mẫu dương tính thực tế.

4.3.5 Độ chính xác (Precision)

Precision là một chỉ số được sử dụng trong các bài toán phân loại để đánh giá độ chính xác của các dự đoán dương tính. Toán học, precision được định nghĩa như sau:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Trong đó:

- TP (True Positives) là số lượng mẫu dương tính được dự đoán đúng.
- FP (False Positives) là số lượng mẫu âm tính bị dự đoán sai thành dương tính.

Chỉ số precision cho biết tỷ lệ các mẫu được dự đoán là dương tính mà thực sự là dương tính trong tổng số các mẫu được dự đoán là dương tính.

4.3.6 F1-score

F1-score là một chỉ số được sử dụng trong các bài toán phân loại để đánh giá hiệu suất của mô hình, kết hợp cả chỉ số precision và recall. Toán học, F1-score được định nghĩa như sau:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trong đó:

- Precision là độ chính xác của các dự đoán dương tính.
- Recall là khả năng nhận diện chính xác các mẫu dương tính.

Chỉ số F1-score cho chúng ta một độ đo hài hòa giữa precision và recall, đặc biệt hữu ích trong các bài toán mà có sự mất cân bằng giữa các lớp.

Sau khi đã nắm được các kiến thức về các chỉ số đánh giá mô hình, chúng ta sẽ đánh giá các mô hình đã huấn luyện dựa vào các chỉ số này.

4.3.7 Intersection over union (IoU)

Tỉ lệ giao hợp (IoU) là một phép đo quan trọng trong phát hiện đối tượng, cung cấp một đo lường định lượng về mức độ chồng chéo giữa một hộp giới hạn thực tế (ground truth - gt) và một hộp giới hạn dự đoán (predicted - pd) được sinh ra bởi bộ phát hiện đối tượng. Phép đo này là cơ sở để đánh giá độ chính xác của các mô hình phát hiện đối tượng và được sử dụng để định nghĩa các thuật ngữ quan trọng như True Positive (TP), False Positive (FP), và False Negative (FN).

Công thức của IoU có thể được biểu diễn như sau trong LaTeX:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Trong đó:

- Area of Overlap là diện tích phần giao giữa hộp giới hạn thực tế và hộp giới hạn dự đoán.
- Area of Union là diện tích phần hợp của cả hai hộp giới hạn.

Định nghĩa các thuật ngữ:

- **True Positive (TP)** — Phát hiện chính xác được thực hiện bởi mô hình.
- **False Positive (FP)** — Phát hiện sai được thực hiện bởi bộ phát hiện.

- **False Negative (FN)** — Một Ground-truth bị bỏ sót (không được phát hiện) bởi bộ phát hiện đối tượng.
- **True Negative (TN)** — Vùng nền được phát hiện đúng là không phát hiện bởi mô hình. Thông số này không được sử dụng trong phát hiện đối tượng vì các vùng này không được chú thích rõ ràng khi chuẩn bị các chú thích.

4.3.8 Độ chính xác trung bình (mAP)

Độ chính xác trung bình (mAP) là một phép đo quan trọng trong phát hiện đối tượng đánh giá hiệu suất của mô hình bằng cách xem xét cả độ chính xác và độ phủ của nhiều lớp đối tượng. Cụ thể, mAP50 tập trung vào ngưỡng IoU là 0.5, đo lường khả năng của mô hình xác định đối tượng có sự chồng chéo hợp lý. Điểm mAP50 cao hơn cho thấy hiệu suất tổng thể tốt hơn.

Để cung cấp một đánh giá toàn diện hơn, mAP50-95 mở rộng đánh giá đến một loạt ngưỡng IoU từ 0.5 đến 0.95. Thước đo này đặc biệt quan trọng đối với các nhiệm vụ yêu cầu định vị chính xác và phát hiện đối tượng chi tiết.

Trong thực tế, mAP50 và mAP50-95 giúp đánh giá hiệu suất mô hình qua các lớp và điều kiện khác nhau, mang lại thông tin về độ chính xác của phát hiện đối tượng trong khi xem xét sự đánh đổi giữa độ chính xác và độ phủ. Các mô hình có điểm mAP50 và mAP50-95 cao hơn được coi là đáng tin cậy và phù hợp cho các ứng dụng đòi hỏi cao như lái xe tự động và giám sát an ninh.

$$\text{mAP} = \frac{1}{|\text{classes}|} \sum_{i \in \text{classes}} \text{AP}_i$$

Trong đó:

- $|\text{classes}|$ là số lượng lớp đối tượng.
- $\text{AP}(i)$ là Precision trung bình tại ngưỡng IoU i .

4.4 Kết quả huấn luyện mô hình bình phương tối thiểu riêng phần (PLSR)

Mô hình bình phương tối thiểu riêng phần dùng để dự đoán nồng độ hóa chất AAA dựa vào tín hiệu điện Raman trong nuôi vi tảo trong công nghiệp và được sử dụng để tích hợp vào hệ thống giám sát nồng độ hóa chất AAA sinh ra trong nuôi vi tảo thời gian thực. Việc tích hợp các thuật toán máy học và học sâu hay trí tuệ nhân tạo vào các hệ thống thời gian thực giúp số hóa trong sản xuất và trong công nghiệp.

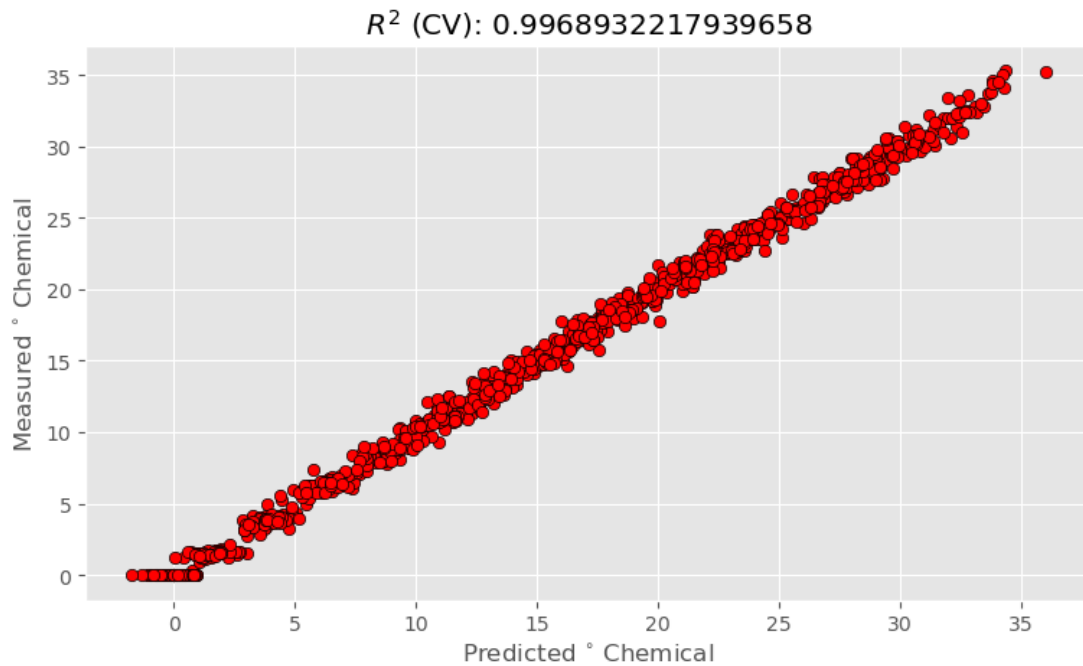
Trong các bài toán hồi quy tuyến tính, chúng ta thường sử dụng các chỉ số như R-square, MSE, MAE để đánh giá kết quả mô hình dự đoán so với giá trị thực tế. Những giá trị trên giúp chúng ta đánh giá về mô hình đã huấn luyện trong khả năng dự đoán dữ liệu trong thực tế.

Mô hình	R^2	MSE	MAE
PLSR	0.9968	0.327	0.459

Bảng 4.1: Kết quả mô hình.

Mô hình PLSR có chỉ số R^2 gần bằng 1 cho thấy mô hình giải thích tốt phần lớn sự biến thiên của dữ liệu, tức là mô hình rất phù hợp với dữ liệu. Ngoài ra, MSE và MAE của mô hình đều thấp, cho biết mức độ lỗi trung bình bình phương và trung bình tuyệt đối giữa giá trị dự

đoán gần với giá trị thực tế. Dựa trên kết quả này, mô hình PLSR có đạt khả năng tốt trong việc dự đoán và giải thích biến động dữ liệu.



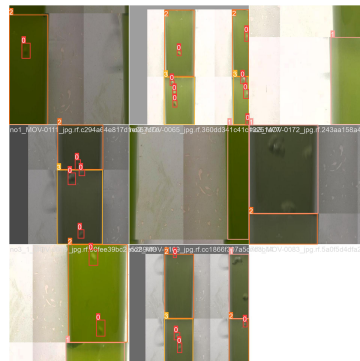
Hình 4.4: Giá trị thực tế và dự đoán trên tập dữ liệu test.

Giá trị dự đoán và giá trị thực tế trên tập dữ liệu kiểm tra gần bằng nhau, mô hình có thể dự đoán tốt.

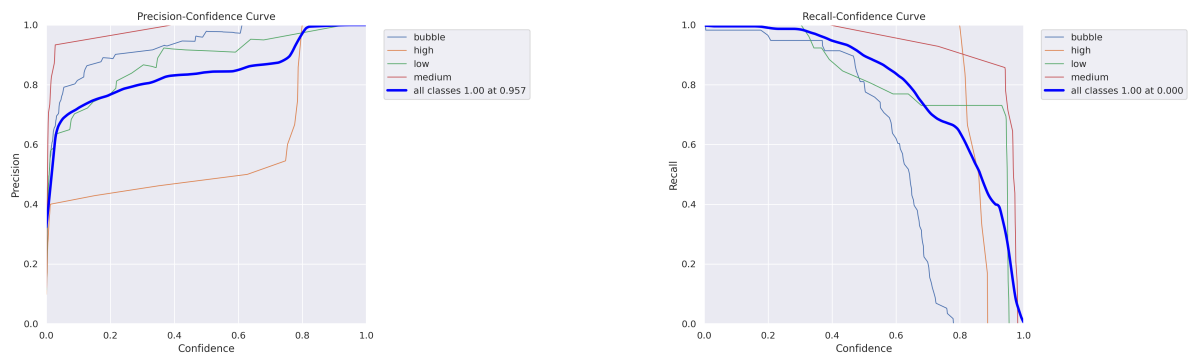
Tiếp theo, chúng tôi sẽ trình bày kết quả huấn luyện mô hình trên tập dữ liệu ảnh bọt khí bằng YOLOv8

4.5 Kết quả huấn luyện mô hình YOLOv8

Chúng tôi đã thực hiện huấn luyện mô hình YOLOv8 trên tập dữ liệu ảnh bọt khí mô phỏng trong nuôi vi tảo để nhận diện và phân loại các bọt khí. Dưới đây là các chỉ số chính thu được sau quá trình huấn luyện:



Hình 4.5: Quá trình huấn luyện dữ liệu.



Hình 4.6: Precision và Recall.

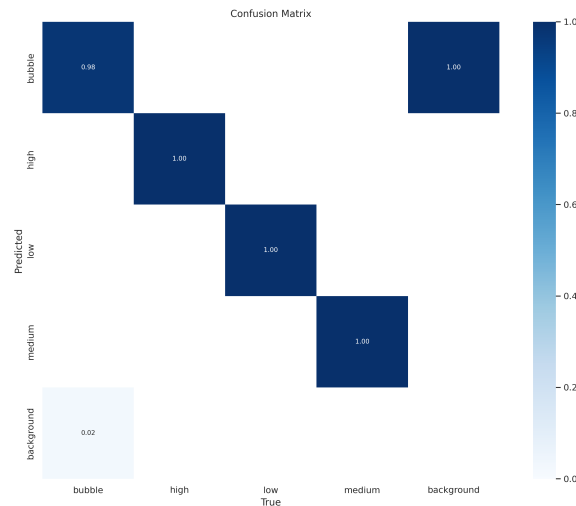
Biểu đồ Recall-Confidence (biểu đồ độ nhạy và độ tin cậy) cho thấy mối quan hệ giữa độ tin cậy của các dự đoán và độ nhạy của mô hình cho từng lớp đối tượng cũng như tất cả các lớp đối tượng.

- Bubble: Đường cong của lớp "bubble" giảm đáng kể khi độ tin cậy vượt quá ngưỡng 0.6. Điều này cho thấy rằng khi mô hình yêu cầu dự đoán với độ tin cậy cao hơn, độ nhạy (recall) của lớp "bubble" giảm mạnh. Điều này có thể chỉ ra rằng mô hình khó khăn hơn trong việc dự đoán các bọt khí với độ tin cậy cao.
- Các lớp "high", "low", và "medium" đều có đường cong recall tương đối ổn định cho đến khi đạt độ tin cậy cao gần 1.0. Các đường cong này cho thấy rằng các lớp này giữ được độ nhạy tốt hơn so với lớp "bubble" ngay cả khi độ tin cậy yêu cầu cao hơn.

Biểu đồ Precision-Confidence (biểu đồ độ chính xác và độ tin cậy) cho thấy mối quan hệ giữa độ tin cậy của các dự đoán và độ chính xác của mô hình cho từng lớp đối tượng cũng như tất cả các lớp đối tượng.

- Bubble: Đường cong của lớp "bubble" tăng dần và đạt độ chính xác cao khi độ tin cậy gần đạt 0.8. Độ chính xác của lớp "bubble" bắt đầu ở mức thấp và tăng dần theo độ tin cậy, điều này cho thấy rằng khi mô hình yêu cầu độ tin cậy cao hơn, độ chính xác của dự đoán tăng lên rõ rệt.
- Các lớp "high", "low", và "medium" đều có đường cong độ chính xác khá cao từ sớm và duy trì ổn định khi độ tin cậy tăng lên. Điều này cho thấy rằng các lớp này duy trì độ chính xác cao ngay cả ở các mức độ tin cậy thấp.

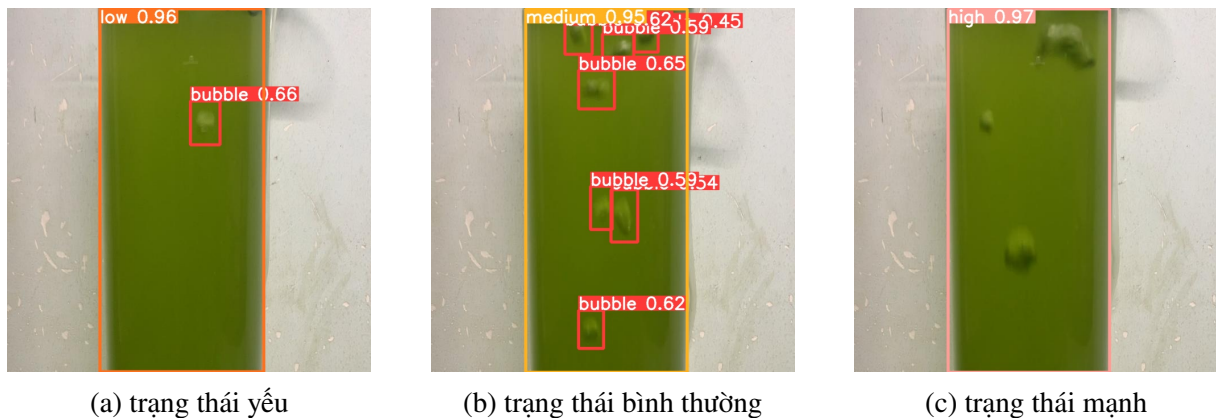
Biểu đồ ma trận confusion matrix cho thấy hiệu suất của mô hình YOLOv8 trong việc phân loại các đối tượng vào các lớp cụ thể. Ma trận này cung cấp thông tin chi tiết về số lượng dự đoán chính xác và sai lệch của mô hình cho từng lớp đối tượng.



Hình 4.7: Confution matrix.

Tỉ lệ dự đoán chính xác là 0.98, nghĩa là 98% các đối tượng thuộc lớp "bubble" được phân loại chính xác. Tất cả các đối tượng thuộc các lớp "high", "low", và "medium" đều được phân loại chính xác, với tỉ lệ chính xác 100%. Có một tỉ lệ nhỏ 0.02 các đối tượng nền (background) bị phân loại nhầm thành lớp "bubble".

Dưới đây là một số hình ảnh kết quả minh họa quá trình nhận diện bọt khí bởi mô hình YOLOv8:



Hình 4.8: Kết quả dự đoán 3 trạng thái trong bộ dữ liệu.

Mô hình của chúng tôi đã dự đoán chính xác bọt khí và phân loại chất lượng bọt khí với độ chính xác cao. Đối với trạng thái yếu (low) và trạng thái bình thường (medium), mô hình đếm số lượng bọt khí (bubble) trên từng khung hình. Như vậy, chúng tôi đã thực hiện mô phỏng lại quá trình nuôi vi tảo trong công nghiệp và thực hiện thử nghiệm mô hình trên tập dữ liệu mô phỏng với kết quả đầu ra khá tốt, do đó chúng tôi hoàn toàn có thể áp dụng mô hình này vào hệ thống phát hiện bọt khí thời gian thực giúp cho việc theo dõi chất lượng nuôi vi tảo tự động và hiệu quả hơn.

So sánh hiệu suất giữa YOLOv8 và các mô hình khác

Mô hình	P	R	mAP50	mAP50-95
YOLOv5	0.973	0.979	0.989	0.83
YOLOv7	0.977	0.987	0.997	0.841
YOLOv8	0.984	0.987	0.993	0.855

Bảng 4.2: So sánh hiệu suất các mô hình nhận diện.

YOLOv5 đạt được chỉ số mAP50 rất cao (0.989), cho thấy khả năng chính xác trong việc phát hiện đối tượng. Tuy nhiên, chỉ số mAP50-95 là 0.83, mặc dù vẫn rất tốt, nhưng thấp hơn so với các phiên bản YOLO sau này. YOLOv7 cải thiện so với YOLOv5 ở mọi chỉ số, đặc biệt là ở mAP50 (0.997) và mAP50-95 (0.841). Điều này cho thấy YOLOv7 có độ chính xác cao hơn và khả năng phát hiện đối tượng tốt hơn trong các điều kiện đa dạng hơn. YOLOv8 tiếp tục cải thiện so với YOLOv7, đặc biệt ở chỉ số mAP50-95 (0.855), cho thấy khả năng phát hiện đối tượng tốt hơn trong nhiều điều kiện khác nhau. Precision cũng tăng lên 0.984, mặc dù mAP50 có giảm nhẹ so với YOLOv7 nhưng vẫn ở mức rất cao (0.993).

Cả ba phiên bản YOLO đều có hiệu suất ấn tượng, nhưng YOLOv8 rõ ràng là phiên bản cải tiến nhất với độ chính xác cao nhất và khả năng phát hiện đối tượng tốt nhất trong các điều kiện đa dạng. YOLOv7 cũng là một bước tiến lớn so với YOLOv5, với các chỉ số đều vượt trội hơn.

Chương 5

Kết luận

Trong chương này, chúng tôi sẽ tổng kết lại toàn bộ công trình nghiên cứu, bao gồm việc nhấn mạnh vào mục tiêu nghiên cứu, kết quả đạt được, đóng góp chính, cũng như phân tích các thử nghiệm đã thực hiện. Chúng tôi cũng sẽ đề cập đến những hạn chế của nghiên cứu và đề xuất hướng phát triển trong tương lai. Cuối cùng, chúng tôi sẽ nhấn mạnh tầm quan trọng và ứng dụng thực tiễn của công trình nghiên cứu này.

5.1 Tóm tắt mục tiêu nghiên cứu

Mục tiêu chính của nghiên cứu này là xây dựng một hệ thống hoàn chỉnh về áp dụng trí tuệ nhân tạo, máy học và học sâu vào theo dõi và giám sát nuôi vi tảo trong công nghiệp thời gian thực. Để đạt được mục tiêu này, chúng tôi đã tiến hành thu thập dữ liệu về tín hiệu Raman và dữ liệu về bọt khí trong nuôi vi tảo, tự lắp đặt mô phỏng lại quá trình sục khí CO₂ trong nuôi vi tảo, tiến hành huấn luyện các mô hình cho từng bộ dữ liệu trên và thu được kết quả tốt có thể áp dụng. Sau đó, chúng tôi tiến hành tìm hiểu các công nghệ dùng để triển khai hệ thống thời gian thực có thể tích hợp các mô hình máy học và học sâu vào để thực hiện cho mục tiêu tối ưu hóa và số hóa trong nuôi vi tảo trong công nghiệp.

5.2 Kết quả và đóng góp chính

Nghiên cứu đã đạt được các kết quả quan trọng sau: Mô hình bình phương tối thiểu riêng phần dùng để huấn luyện tập dữ liệu Raman dùng để dự đoán nồng độ chất AAA trong quá trình nuôi vi tảo cho kết quả tốt với các chỉ số đánh giá mô hình đạt và cho thấy mô hình có thể ứng dụng; mô hình Yolov8 dùng để huấn luyện bộ dữ liệu bọt khí trong nuôi vi tảo với các chỉ số đánh giá mô hình khá tốt và nhận diện chính xác bọt khí cũng như các trạng thái môi trường trong quá trình nuôi vi tảo; chúng tôi đã xây dựng thành công 2 hệ thống xử lý dữ liệu thời gian thực có ứng dụng hệ thống điện toán đám mây, máy học và học sâu vào việc theo dõi và giám sát nồng độ hóa chất AAA và kiểm soát chất lượng sục bọt khí CO₂ trong quá trình nuôi vi tảo, chúng tôi đã tìm hiểu và trình bày chi tiết kiến trúc và nguyên lý hoạt động từng công nghệ đã sử dụng trong bài báo cáo, bằng việc hiểu rõ cách thức hoạt động chúng ta sẽ dễ dàng kiểm soát và dễ dàng vận hành hệ thống. Những kết quả này không chỉ giúp làm sáng tỏ việc ứng dụng điện toán đám mây, máy học và học sâu vào nuôi vi tảo mang lại hiệu quả cao, mà còn đóng góp vào việc số hóa công nghiệp.

5.3 Những thử nghiệm và phân tích

Trong quá trình nghiên cứu, chúng tôi đã thực hiện một loạt các thử nghiệm để tìm ra mô hình tốt nhất, các công nghệ phù hợp. Phân tích kết quả cho thấy 2 mô hình đã trình bày cho kết quả khá tốt và hệ thống vận hành tốt, bền bỉ đáp ứng yêu cầu về tốc độ và độ chính xác, điều này chứng minh những thử nghiệm của chúng tôi là chính xác với chủ đề nghiên cứu này.

5.4 Những hạn chế và hướng phát triển

Mặc dù nghiên cứu đã đạt được nhiều kết quả đáng kể, nhưng vẫn còn một số hạn chế như không có dữ liệu thực tế, chi phí vận hành còn cao, tốc độ và độ chính xác của dự đoán khá tốt nhưng cần tinh chỉnh thêm. Trong tương lai, chúng tôi dự định tiếp tục thử nghiệm các mô hình trên tập dữ liệu thực tế và tìm cách áp dụng các công nghệ mã nguồn mở cũng với điện toán đám mây để khắc phục những hạn chế này và mở rộng nghiên cứu.

5.5 Tầm quan trọng và ứng dụng thực tiễn

Công trình nghiên cứu này có tầm quan trọng đặc biệt trong nuôi vi tảo trong công nghiệp, với khả năng theo dõi và giám sát toàn diện thời gian thực quá trình sinh ra chất AAA và quá trình sục khí CO₂. Ứng dụng thực tiễn của nghiên cứu bao gồm theo dõi và dự đoán nồng độ hóa chất AAA sinh ra trong quá trình nuôi vi tảo và giám sát chất lượng bọt khí CO₂ trong quá trình sục khí CO₂ vào vi tảo, đồng thời cảnh báo những trường hợp sục khí CO₂ quá mạnh hoặc quá yếu, góp phần vào việc tối ưu quá trình nuôi vi tảo trong công nghiệp và số hóa công nghiệp.

Tổng kết lại, công trình nghiên cứu này không chỉ mở ra những triển vọng mới trong việc nuôi vi tảo trong công nghiệp mà còn đề xuất các hướng phát triển hứa hẹn cho tương lai, hướng tới việc giải quyết theo dõi và giám sát trong công nghiệp một cách hiệu quả hơn.

Tài liệu tham khảo

- [1] Svante Wold, Michael Sjöström, Lennart Eriksson, PLS-regression: a basic tool of chemometrics, 2001.
- [2] Dillon Reis, Jordan Kupec, Jacqueline Hong, Ahmad Daoudi Georgia Institute of Technology, Real-Time Flying Object Detection with YOLOv8, 2023.
<https://www.semanticscholar.org/reader/231a434f8fac0b01cbc05890b283f4d9da4cb100>
- [3] Wold, S., Geladi, P., Esbensen, K., Öhman, J., 1987. Multi-way principal components and PLS-analysis. *J. Chemom.* 1 (1), 41-56.
<https://www.sciencedirect.com/science/article/abs/pii/S0959152408001558>
- [4] Francis Chinomso Maduakolam, *Samson Dauda Yusuf, Ibrahim Umar, Abdullahi Abubakar Mund. Analysis of Savitzky-Golay Filter for Electrocardiogram De-Noising Using Daubechies Wavelets.
<https://www.semanticscholar.org/paper/Analysis-of-Savitzky-Golay-Filter-for-De-75b43957b3cab43da3e92ab38679c236f6c63a28>
- [5] <https://medium.com/@techlakshmi.india/navigating-apache-delta-lake-architecture/>
- [6] <https://learn.microsoft.com/en-us/azure/event-hubs/event-hubs-about>
- [7] Wang, S.; Gao, S.; Zhou, L.; Liu, R.; Zhang, H.; Liu, J.; Jia, Y.; Qian, J. YOLO-SD: Small Ship Detection in SAR Images by Multi-Scale Convolution and Feature Transformer Module.
- [8] <https://grafana.com/docs/agent/latest/operator/architecture/>
- [9] Zhang, Z.; Lu, X.; Cao, G.; Yang, Y.; Jiao, L.; Liu, F. ViT-YOLO: Transformer-based YOLO for object detection. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, BC, Canada, 11–17 October 2021; pp. 2799–2808.
- [10] Gang Wang, Yanfei Chen *, Pei An, Hanyu Hong, Jinghu Hu and Tiange Huang, UAV-YOLOv8: A Small-Object-Detection Model Based on Improved YOLOv8 for UAV Aerial Photography Scenarios, MPDI, 2023.
- [11] Dillon Reis*, Jacqueline Hong, Jordan Kupec, Ahmad Daoudi Georgia Institute of Technology, Real-Time Flying Object Detection with YOLOv8, 2024.
- [12] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-iou loss: Faster and better learning for bounding box regression. *CoRR*, abs/1911.08287, 2019.
- [13] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. *CoRR*, abs/2006.04388, 2020.
- [14] Tausif Diwan¹ G. Anirudh² Jitendra V. Tembhurne¹, Object detection using YOLO: challenges, architectural successors, datasets and applications, 2022.

-
- [15] Chung T, Ball S, Stentz A (2018) Early fire detection using machine learning in smart buildings. In: Proceedings of the 21st international conference on information fusion (FUSION), pp 1–8.
 - [16] Avazov K et al (2021) Fire detection method in smart city environments using a deep-learning-based approach.
 - [17] Li T, Zhao E, Zhang J, Hu C (2019) Detection of wildfire smoke images based on a densely dilated convolutional network.
 - [18] Muhammad K, Ahmad J, Mehmood I, Rho S, Baik SW (2018) Convolutional neural networks based fire detection in surveillance videos.
 - [19] Li P, Zhao W (2020) Image fire detection algorithms based on convolutional neural networks.
 - [20] Biswas A, Ghosh SK, Ghosh A (2023) Early fire detection and alert system using modified inception-v3 under deep learning framework.
 - [21] Khan A et al (2022) CNN-based smoker classification and detection in smart city application.
 - [22] Wang F, Li J, Li Y, Li Y, Huang Y (2018) Real-time fire detection in surveillance video using YOLOv2. In: 2018 13th IEEE conference on industrial electronics and applications (ICIEA), pp 2428–2432.
 - [23] Jia J, Cao Y, Wang X, Huang J (2019) Fire detection based on multi-model fusion. In: 2019 4th International conference on image, vision and computing (ICIVC), pp 329–332.
 - [24] Huang J, Luo Q, Wang J, Guo J (2020) Fire detection in outdoor scenes using YOLOv3.
 - [25] Al-Turjman F, Al-Karaki JN, Al-Bzoor Z (2021) Hybrid deep learning-based approach for fire detection in smart cities.
 - [26] Gohari A et al (2022) Involvement of surveillance drones in smart cities: a systematic review.
 - [27] Zhang F, Zhao P, Xu S, Wu Y, Yang X, Zhang Y (2020) Integrating multiple factors to optimize watchtower deployment for wildfire detection.
 - [28] Huang PY, Chen YT, Wu CC (2019) A fire detection system for smart buildings based on deep learning.
 - [29] Chung T, Ball S, Stentz A (2018) Early fire detection using machine learning in smart buildings. In: Proceedings of the 21st international conference on information fusion.
 - [30] Abdusalomov AB et al (2023) An improved forest fire detection method based on the detectron2 model and a deep learning approach.
 - [31] Xia, G.S.; Bai, X.; Ding, J.; Zhu, Z.; Belongie, S.; Luo, J.; Datcu, M.; Pelillo, M.; Zhang, L. DOTA: A large-scale dataset for object detection in aerial images. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–22 June 2018; pp. 3974–3983.
 - [32] Zhang, Z.; Lu, X.; Cao, G.; Yang, Y.; Jiao, L.; Liu, F. ViT-YOLO: Transformer-based YOLO for object detection. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, BC, Canada, 11–17 October 2021; pp. 2799–2808.
 - [33] Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; Zagoruyko, S. End-to-end object detection with transformers. In Proceedings of the European Conference on Computer Vision, Glasgow, UK, 23–28 August 2020; pp. 213–229
 - [34] Fang, Y.; Liao, B.; Wang, X.; Fang, J.; Qi, J.; Wu, R.; Niu, J.; Liu, W. You only look at one sequence: Rethinking transformer in vision through object detection. *Adv. Neural Inf. Process. Syst.* 2021, 34, 26183–26197.
-

-
- [35] Shao, S.; Li, Z.; Zhang, T.; Peng, C.; Yu, G.; Zhang, X.; Li, J.; Sun, J. Objects365: A large-scale, high-quality dataset for object detection. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Seoul, Republic of Korea, 27 October–2 November 2019; pp. 8430–8439.
 - [36] Xu, S.; Wang, X.; Lv, W.; Chang, Q.; Cui, C.; Deng, K.; Wang, G.; Dang, Q.; Wei, S.; Du, Y.; et al. PP-YOLOE: An evolved version of YOLO. arXiv 2022, arXiv:2203.16250.
 - [37] Liu, R.; Lehman, J.; Molino, P.; Petroski Such, F.; Frank, E.; Sergeev, A.; Yosinski, J. An intriguing failing of convolutional neural networks and the coordconv solution. In Proceedings of the 32nd Conference on Neural Information Processing Systems (NeurIPS 2018), Montréal, QC, Canada, 3–8 December 2018; pp. 9628–9639.
 - [38] Wang, C.Y.; Bochkovskiy, A.; Liao, H.Y.M. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. arXiv 2022, arXiv:2207.02696.
 - [39] Hussain, M. YOLO-v1 to YOLO-v8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection. *Machines* 2023, 11, 677.

Phụ lục

Phụ lục A. Mã mô phỏng gửi dữ liệu Raman

```
19
20 producer = EventHubProducerClient.from_connection_string(conn_str=event_hub_connection_st
21 Cust='ABC'
22 Project_ID='ABC-01'
23 BatchID = 1
24
25 data = pd.read_csv('Offline_raman1.csv')
26 data.insert(0,'Cust',Cust,True)
27 data.insert(1,'Project_ID',Project_ID,True)
28 data.insert(2,'BatchID',BatchID,True)
29
30 i = 0
31 df_json = data.apply(lambda x: x.to_json(), axis=1)
32 N = df_json.shape[0]
33 Scan = 0
34 try:
35     for i in df_json.index:
36         event_data_batch = producer.create_batch() # Create a batch. You will add events
37         print('Start send .....')
38         print()
39         # s = json.dumps(df_json[i]) # Convert the reading into a JSON string.
40         # print(s)
41         Scan += 1
42         current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
43         event_data_with_time = json.loads(df_json[i])
44         event_data_with_time = {'Scan': Scan, **event_data_with_time}
45         event_data_with_time = {'Time stream': current_time, **event_data_with_time}
46
47
48         print(event_data_with_time)
49         event_data_batch.add(EventData(json.dumps(event_data_with_time))) # Add event dat
50         producer.send_batch(event_data_batch)
51         time.sleep(1) #delate every 10s
52 except KeyboardInterrupt:
53     #pass
54     producer.close()
55
```

NORMAL generate_realtime_eventhub_raman.py % main

Hình 5.1: Mô phỏng gửi dữ liệu Raman.

Phụ lục B. Triển khai mô hình máy học

Default Directory > MLV2 > Endpoints > raman-pls-service-v2-5

raman-pls-service-v2-5 ☆

Details

Test

Consume

Logs

Input data to test endpoint

Test

```
{
  "Inputs": [{"1300": 0.0, "1299": 0.0, "1298": 0.0, "1297": 0.0, "1296": 0.0, "1295": 0.0, "1294": 0.0, "1293": 0.0, "1292": 0.0, "1291": 0.0, "1290": 0.0, "1289": 0.0, "1288": 0.0, "1287": 0.0, "1286": 0.0, "1285": 0.0, "1284": 0.0, "1283": 0.0, "1282": 0.0, "1281": 0.0, "1280": 0.0, "1279": 0.0, "1278": 0.0, "1277": 0.0, "1276": 0.0, "1275": 0.0, "1274": 0.0, "1273": 0.0, "1272": 0.0, "1271": 0.0, "1270": 0.0, "1269": 0.0, "1268": 0.0, "1267": 0.0, "1266": 0.0, "1265": 0.0, "1264": 0.0, "1263": 0.0, "1262": 0.0, "1261": 0.0, "1260": 0.0, "1259": 0.0, "1258": 0.0, "1257": 0.0, "1256": 0.0, "1255": 0.0, "1254": 0.0, "1253": 0.0, "1252": 0.0, "1251": 0.0, "1250": 0.0, "1249": 0.0, "1248": 0.0, "1247": 0.0, "1246": 0.0, "1245": 0.0, "1244": 0.0, "1243": 0.0, "1242": 0.0, "1241": 0.0, "1240": 0.0, "1239": 0.0, "1238": 0.0, "1237": 0.0, "1236": 0.0, "1235": 0.0, "1234": 0.0, "1233": 0.0, "1232": 0.0, "1231": 0.0, "1230": 0.0, "1229": 0.0, "1228": 0.0, "1227": 0.0, "1226": 0.0, "1225": 0.0, "1224": 0.0, "1223": 0.0, "1222": 0.0, "1221": 0.0}]]
```

Test result

```
[ 1 item
  0: { 1 item
    "Predict_Pen": float -0.0011450364281877512
  }
]
```

Hình 5.2: Tạo điểm cuối của mô hình dữ liệu.